

COMSATS University Islamabad

Lahore Campus

Department of Computer Science

FINAL YEAR PROJECT MID REPORT

Session 2023 – 2027

DocMate: Smart Healthcare Appointment System

Submitted in partial fulfilment of the requirements for the Degree of
Bachelor of Science in Computer Science (BS CS)

Sr. #	Reg. No.	Student Name	Email ID
(i)	FA23-BCS-XXX	Hamza Naseem	hamza@cuilahore.edu.pk
(ii)	FA23-BCS-XXX	Asfand Yar Khan	asfand@cuilahore.edu.pk

Role	Name	Signature
Project Supervisor	[Supervisor Name]	_____
Head of Department	[HoD Name]	_____

Date of Submission: _____

Declaration

We, the undersigned, hereby declare that the work presented in this Final Year Project report titled DocMate: Smart Healthcare Appointment System is our own original work conducted under the supervision of [Supervisor Name] in the Department of Computer Science, COMSATS University Islamabad, Lahore Campus.

We confirm that:

- This report has not been submitted elsewhere for any academic award or qualification.
- The project work is free from plagiarism. The similarity index is less than 20% as certified by the department.
- All sources of information used in this report have been duly acknowledged and cited.
- The experimental results and data presented herein are genuine and unaltered.
- All intellectual contributions by team members have been appropriately recognised.

Sr. #	Name	Reg. No.	Signature	Date
(i)	Hamza Naseem	FA23-BCS-XXX	_____	_____
(ii)	Asfand Yar Khan	FA23-BCS-XXX	_____	_____

Supervisor Approval:

Supervisor	Signature	Date
[Supervisor Name]	_____	_____

Acknowledgement

We are deeply grateful to Almighty Allah for granting us the strength, patience, and ability to complete this phase of our Final Year Project successfully.

We would like to express our sincere gratitude to our respected project supervisor, [Supervisor Name], for his/her continuous guidance, valuable suggestions, and constructive feedback throughout the development of this project. His/Her support has played a vital role in shaping both the technical and research dimensions of our work.

We are also thankful to the faculty members of the Department of Computer Science, COMSATS University Islamabad, Lahore Campus, for providing us with the academic environment, knowledge, and motivation necessary to undertake this project. Their teachings throughout our degree have contributed significantly to our understanding of software engineering, artificial intelligence, and system design principles.

We would also like to acknowledge the support and cooperation of our classmates, friends, and peers who provided useful discussions, encouragement, and feedback during various stages of the project. Their insights helped us refine several aspects of the system.

Finally, we are especially thankful to our parents and families for their constant prayers, moral support, and encouragement. Without their trust and motivation, this work would not have been possible.

Abstract

The healthcare sector worldwide is undergoing a fundamental transformation driven by the rapid advancement of artificial intelligence (AI), machine learning (ML), and digital communication technologies. Despite these advances, a large segment of the population continues to face challenges such as prolonged waiting times, inefficient appointment scheduling, fragmented health records, and limited access to specialist consultations — particularly in developing regions.

DocMate: Smart Healthcare Appointment System is a comprehensive, AI-powered web-based healthcare platform designed to bridge the gap between patients and healthcare providers. The system leverages Natural Language Processing (NLP) to analyse patient-reported symptoms and provide intelligent doctor recommendations, eliminating the need for time-consuming and often inefficient triage processes. By integrating an NLP-driven chatbot using Retrieval-Augmented Generation (RAG), DocMate delivers contextually accurate medical guidance sourced from curated medical knowledge bases.

The platform provides real-time video consultations powered by WebRTC technology, enabling remote doctor-patient interactions with minimal latency. Patients can seamlessly book, reschedule, or cancel appointments through an intuitive interface that displays available time slots in real time. The system features dedicated portals for patients, doctors, and administrators — each tailored to the respective user's workflow needs.

DocMate includes a structured electronic health record (EHR) system that stores patients' complete medical histories, past prescriptions, diagnostic notes, and health timelines in an encrypted, role-based access-controlled environment. A medicine information module provides detailed data on commonly prescribed drugs, including dosage guidelines, contraindications, and known side effects.

An emergency alert subsystem detects high-risk symptom patterns reported through the chatbot and automatically notifies emergency contacts and available on-duty physicians. Medication reminders help patients maintain adherence to prescribed treatment schedules.

The technology stack comprises React.js for the frontend, Flask for the backend REST API, MySQL for relational data storage, Firebase for authentication and real-time notifications, and WebRTC/Twilio for audio-video communication. Agile (Scrum) methodology governs the development lifecycle.

DocMate directly addresses United Nations Sustainable Development Goal 3: Good Health and Well-Being by improving healthcare accessibility, reducing friction in patient-provider interactions, and empowering individuals with AI-driven medical information.

Keywords: Artificial Intelligence, Natural Language Processing, Healthcare Management System, Telemedicine, WebRTC, RAG, Appointment Scheduling, Electronic Health Records, Chatbot, Firebase.

Executive Summary

The healthcare sector is increasingly shifting toward digital platforms that can improve accessibility, efficiency, and continuity of care. In developing countries such as Pakistan, patients often experience long waiting times, limited access to specialists, fragmented medical records, and difficulties in obtaining timely healthcare guidance. These issues are further intensified in rural and semi-urban areas where specialist availability and organised appointment systems are limited. In this context, the need for an integrated, technology-driven healthcare solution has become more important than ever.

DocMate: Smart Healthcare Appointment System is proposed as a web-based, AI-enabled healthcare platform designed to simplify and improve the interaction between patients, doctors, and healthcare administrators. The system combines several important healthcare functions within a single unified environment, including intelligent symptom-based assistance, doctor recommendation, online appointment booking, real-time video consultation, electronic health records management, medicine information lookup, and emergency alert support.

A key feature of DocMate is its AI-powered chatbot, which uses Natural Language Processing (NLP) and Retrieval-Augmented Generation (RAG) to analyse patient-entered symptoms and provide context-aware healthcare guidance. Rather than functioning as a diagnostic tool, the chatbot is designed as an assistive triage mechanism that helps users identify the most relevant medical specialty and obtain preliminary information from trusted medical sources. This feature aims to reduce confusion, save time, and support more efficient appointment scheduling.

The project adopts a full-stack architecture using React.js for the frontend, Flask for backend API development, MySQL for relational data storage, Firebase for authentication and real-time services, and WebRTC/Twilio for video consultation. The development process follows the Agile-Scrum methodology, allowing the project to progress in modular and iterative stages. This approach ensures continuous refinement, testing, and integration of system components over time.

At the current mid-report stage, substantial work has already been completed in the areas of literature review, requirements engineering, feasibility analysis, system design, testing strategy, and partial implementation. Core modules such as system architecture, use case modelling, database design, and initial backend/frontend planning have been established. In addition, the early design of the AI chatbot and RAG pipeline indicates the practical feasibility of incorporating intelligent functionality into the system.

From an academic perspective, this project demonstrates the integration of software engineering, human-computer interaction, artificial intelligence, real-time communication, and secure data management in a real-world problem domain. From a societal perspective, DocMate has the potential to improve healthcare accessibility, especially for users who face logistical, informational, or geographical barriers in reaching medical professionals.

Overall, DocMate represents a relevant and impactful healthcare technology solution that aims to modernise patient-provider interaction through digital innovation while maintaining a strong focus on usability, security, and practical applicability.

Abbreviations and Acronyms

Abbreviation	Full Form
AI	Artificial Intelligence
NLP	Natural Language Processing
ML	Machine Learning
RAG	Retrieval-Augmented Generation
API	Application Programming Interface
RBAC	Role-Based Access Control
EHR	Electronic Health Record
WebRTC	Web Real-Time Communication
UI	User Interface
UX	User Experience
SDLC	Software Development Life Cycle
FYP	Final Year Project
CUI	COMSATS University Islamabad
JWT	JSON Web Token
HTTPS	Hypertext Transfer Protocol Secure
REST	Representational State Transfer
CRUD	Create, Read, Update, Delete
ER	Entity-Relationship
SDG	Sustainable Development Goal
CORS	Cross-Origin Resource Sharing
OTP	One-Time Password
SMS	Short Message Service
BERT	Bidirectional Encoder Representations from Transformers
AWS	Amazon Web Services
FAISS	Facebook AI Similarity Search
UMLS	Unified Medical Language System
SDP	Session Description Protocol
ICE	Interactive Connectivity Establishment
STUN	Session Traversal Utilities for NAT
TURN	Traversal Using Relays around NAT
NAT	Network Address Translation
UAT	User Acceptance Testing

CI/CD	Continuous Integration / Continuous Deployment
FCM	Firebase Cloud Messaging
HMIS	Health Management Information System
WHO	World Health Organization
AES	Advanced Encryption Standard
TLS	Transport Layer Security
XSS	Cross-Site Scripting
ORM	Object-Relational Mapping
3NF	Third Normal Form

Chapter 1 – Introduction

1.1 Background

The global healthcare sector is facing unprecedented pressure. Ageing populations, rising chronic disease burdens, and uneven distribution of medical professionals have stretched traditional healthcare delivery models to their limits. According to the World Health Organization (WHO), there is a global shortage of approximately 10 million health workers, a gap that is particularly acute in South Asia and Sub-Saharan Africa. This systemic deficit creates significant barriers to timely and effective care.

Pakistan, with a population exceeding 230 million, has a doctor-to-patient ratio of approximately 1:1,000 — significantly below the WHO-recommended threshold of 1:500. Public hospitals are chronically overcrowded; appointment waiting times can extend to weeks in urban centres and even longer in rural areas, and specialist access in semi-urban and rural regions is virtually non-existent. The result is a deeply two-tiered healthcare system where quality of care is heavily dependent on geography and socioeconomic status.

Digital health technologies have emerged as a viable and increasingly proven strategy to address these systemic inefficiencies. The COVID-19 pandemic served as an unprecedented catalyst for the widespread adoption of telemedicine, remote consultations, and digital health records. Studies estimate that the global digital health market is projected to reach USD 660 billion by 2025, driven by increasing smartphone penetration, improved internet connectivity, and growing patient demand for convenient, affordable healthcare access.

Artificial Intelligence has become a cornerstone of next-generation healthcare applications. Machine learning algorithms have demonstrated accuracy comparable to specialist physicians in tasks such as medical image analysis, drug interaction detection, and symptom triage. Natural Language Processing (NLP) enables conversational AI agents to interpret free-text patient descriptions of symptoms and generate relevant clinical guidance.

Telemedicine platforms such as Teladoc (USA), Practo (India), and Sehat Kahani (Pakistan) have demonstrated the commercial and social viability of remote healthcare delivery. However, these platforms are often fragmented — offering either appointment scheduling or video consultations or records management, but rarely an integrated, AI-augmented combination of all three.

Within this context, DocMate was conceived as a holistic, AI-augmented healthcare appointment and consultation platform. Rather than addressing a single pain point in the patient journey, DocMate integrates intelligent symptom checking, multi-specialty doctor discovery, online appointment scheduling, real-time video consultation, electronic health records management, and pharmacological information into a unified, user-centric platform.

1.2 Problem Statement

The current healthcare appointment and consultation process in Pakistan and similar developing economies suffers from multiple structural and technological deficiencies:

- **Long Waiting Times:** Patients visiting public healthcare facilities routinely wait for several hours before receiving medical attention, even for non-emergency conditions.
- **Lack of Pre-Visit Triage:** Without an intelligent pre-screening system, patients are often directed to the wrong specialist or department, wasting both the patient's and the physician's time.
- **Geographic Barriers to Specialist Access:** Patients residing in semi-urban or rural areas frequently must travel long distances to access specialist consultations, leading to delayed diagnoses.

- **Fragmented Medical Records:** Patient health histories are typically maintained in paper format or across disparate, non-interoperable digital systems.
- **Medication Non-Adherence:** Patients managing chronic conditions often lack reliable information about their prescribed medications and receive inadequate reminders.
- **Mental Health and Specialist Hesitancy:** A significant proportion of patients avoid or delay consultations due to stigma, distance, or financial cost.
- **Administrative Inefficiency in Doctor Portals:** Many physicians lack integrated tools to manage their schedules, access patient records, write digital prescriptions, and conduct follow-ups in a single interface.

1.3 Proposed Solution

DocMate is an AI-powered, full-stack web application that unifies the core components of a modern healthcare engagement workflow into a single platform. The proposed system is architected around three primary user roles — Patient, Doctor, and Administrator — each with a tailored, role-specific portal.

Patient-Facing Interface:

- An NLP-driven chatbot for intelligent symptom analysis and specialist recommendation, backed by a RAG pipeline querying approximately 15,000 medical document chunks.
- An appointment booking module with real-time doctor availability, specialty filtering, and automated confirmation/reminder notifications.
- Real-time video and chat consultations powered by WebRTC technology with TURN relay fallback.
- A comprehensive personal health dashboard displaying past consultations, prescriptions, and diagnoses.
- A medicine information module with searchable drug data.
- An emergency alert system that automatically escalates high-risk symptom reports.

1.4 Main Objectives

1. To design and develop a fully functional AI-powered healthcare appointment and consultation platform accessible via standard web browsers.
2. To implement an NLP chatbot using RAG for intelligent symptom analysis, doctor recommendation, and medical FAQ responses.
3. To provide seamless appointment scheduling with real-time availability management and automated notifications.
4. To enable high-quality real-time audio-video consultations between patients and doctors using WebRTC.
5. To develop a secure Electronic Health Record (EHR) system accessible to authorised users only.
6. To build a medicine information database for drug queries including dosage, contraindications, and side effects.
7. To implement an emergency alert mechanism for high-risk symptom patterns.
8. To design intuitive, role-based user interfaces with responsive design.
9. To apply industry-standard security practices including AES-256 encryption, RBAC, and HTTPS.
10. To contribute to UN SDG 3 (Good Health and Well-Being) through digital innovation.

1.5 Assumptions and Constraints

1.5.1 Assumptions

- Users have access to a reliable internet connection (minimum 1 Mbps standard, 5 Mbps for video consultations).
- Patients possess basic digital literacy.
- Doctors registered on the platform are licensed medical professionals verified by the platform administrator.
- Patients provide accurate information when describing symptoms.
- Firebase Authentication and Firestore services remain available throughout the project lifecycle.
- All medical information in the RAG knowledge base is sourced from reputable, peer-reviewed references.

1.5.2 Constraints

- Timeline: The FYP timeline of approximately 12 months limits feature breadth.
- Budget: Project is developed under student budget constraints using free-tier cloud services where possible.
- Medical Disclaimer: DocMate does not provide medical diagnoses — the chatbot provides guidance only.
- Data Privacy: Full HIPAA compliance is beyond scope but best-practice security measures are implemented.
- Scale: Designed for approximately 50–100 simultaneous users.
- Supported Browsers: Two most recent major versions of Chrome, Firefox, Safari, and Edge.

1.6 Project Scope

1.6.1 In Scope

- Patient registration, login, profile management, and password reset.
- Doctor registration, profile creation, specialty assignment, and schedule management.
- AI-powered chatbot with NLP-based symptom analysis and RAG-based knowledge retrieval.
- Online appointment scheduling with real-time slot display, booking, rescheduling, and cancellation.
- Automated email and SMS notifications for appointments, reminders, and alerts.
- Real-time audio-video consultation via WebRTC with in-consultation chat.
- Patient health records — storage, retrieval, and doctor-side updating.
- Medicine information database — searchable drug reference.
- Emergency alert system — high-risk symptom detection and notification.
- Medication reminder system — scheduled push/email notifications.
- Administrator dashboard — user management, analytics, and configuration.
- Responsive frontend supporting screens from 360px to 1920px width.

1.6.2 Out of Scope

- Medical diagnosis or treatment decision-making.

- Integration with government health registries.
- Mobile native applications (iOS/Android).
- Billing, payment processing, or insurance claim management.
- Wearable device or IoT health sensor integration.
- Laboratory test ordering or results management.

1.6.3 Expected Impact

The proposed DocMate platform is expected to create meaningful academic, social, and technical impact by improving the overall healthcare appointment and consultation workflow. From a patient perspective, the system aims to reduce unnecessary hospital visits, minimise waiting times, and improve access to timely specialist recommendations. By introducing an AI-assisted symptom analysis module, patients can receive informed guidance before visiting a hospital or clinic, thereby improving triage efficiency and reducing confusion in selecting the appropriate doctor.

From the healthcare provider's perspective, DocMate is expected to improve appointment scheduling efficiency, support better management of patient records, and facilitate remote consultations through secure video communication. Doctors will benefit from an integrated environment where scheduling, medical notes, prescriptions, and patient histories are available in one place.

At a broader level, the system contributes to digital transformation in healthcare by encouraging structured record keeping, remote medical support, and intelligent decision assistance. In developing regions such as Pakistan, where healthcare resources are often limited and unevenly distributed, a platform like DocMate can help bridge the accessibility gap between patients and healthcare professionals. Furthermore, the project demonstrates the practical application of artificial intelligence, natural language processing, and real-time communication technologies in solving real-world healthcare challenges.

1.7 Software Development Lifecycle Model

1.7.1 Rationale for Agile-Scrum

The DocMate project adopts the Agile methodology with Scrum as the specific framework. Scrum was selected for the following reasons: iterative development with two-week sprints, flexibility for dynamic requirements, suitability for small cross-functional teams of 2–9 members, continuous testing per sprint, and transparency through sprint planning and retrospectives.

1.7.2 Sprint Plan Overview

Sprint	Duration	Primary Goals	Deliverables
Sprint 1	Feb 1–14, 2026	Requirements analysis, research	SRS Document, Research Report
Sprint 2	Feb 15 – Mar 15	Data collection, system design	System Design Document, Dataset
Sprint 3	Mar 16 – Apr 15	NLP model & frontend (React)	Chatbot prototype, React UI skeleton
Sprint 4	Apr 16 – May 15	RAG pipeline, patient portal	Functional RAG chatbot, Patient dashboard

Sprint 5	May 16 – Jun 15	Backend API (Flask), doctor portal	REST API, Doctor dashboard
Sprint 6	Jun 16 – Jul 15	Firebase auth, WebRTC	Login/auth system, Video consultation module
Sprint 7	Jul 16 – Aug 15	Admin dashboard, medicine DB, EHR	Admin panel, Medicine search, Health records
Sprint 8	Aug 16 – Sep 15	Emergency alerts, integration testing	Emergency module, Notification system
Sprint 9	Sep 16 – Oct 15	System testing, optimisation	Test reports, Optimised build
Sprint 10	Oct 16 – Nov 15	Deployment, documentation	Deployed application, Final report

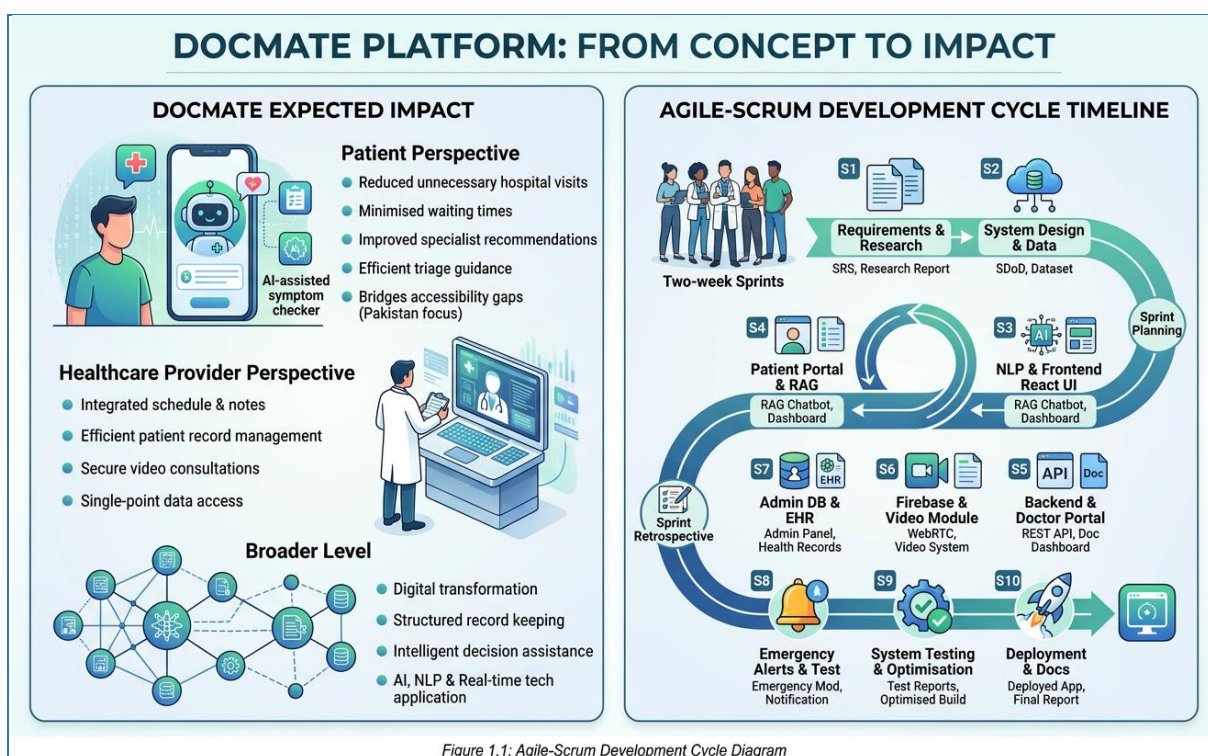


Figure 1.1: Agile-Scrum Development Cycle Diagram

Chapter 2 – Requirement Analysis

2.1 Literature Review

2.1.1 Healthcare Chatbots Using NLP (IEEE, 2022)

This study investigated the use of BERT-based models and Dialogflow for building healthcare chatbots capable of understanding patient symptom descriptions in natural language. The system demonstrated strong intent recognition performance with an F1-score above 0.88 on benchmark medical NLP datasets. DocMate directly builds upon this foundation by augmenting NLP capabilities with a RAG pipeline and embedding the chatbot within a full-stack healthcare management system.

2.1.2 Hospital Management Systems and Scheduling (ACM, 2021)

This research proposed a web-based hospital management system using rule-based scheduling algorithms, achieving a 35% reduction in average waiting times in a pilot hospital in Malaysia. However, it was entirely rule-based and lacked AI components, video consultation, or remote capabilities. DocMate incorporates AI-driven scheduling optimisation and integrates video consultation to address these gaps.

2.1.3 MedAssist and HealthBot Platforms (JMIR, 2021)

MedAssist and HealthBot are cloud-based health record management platforms providing secure access to medical histories with end-to-end encryption and RBAC. Neither offered video consultation, AI chatbot functionality, or intelligent appointment management. DocMate incorporates the secure records philosophy while extending functionality with AI and real-time communication.

2.1.4 WebRTC-Based Telemedicine Platforms (IEEE, 2020)

This research explored WebRTC as a browser-native technology for real-time medical consultations, demonstrating video latency below 300ms. The study validated peer-to-peer encrypted communication for telemedicine without proprietary software installation. DocMate leverages WebRTC for video consultation while adding TURN relay fallback and a comprehensive ecosystem around it.

2.1.5 NLP in Healthcare Applications (Springer, 2022)

This comprehensive review analysed 47 NLP-based healthcare applications, identifying symptom extraction, drug interaction detection, and clinical note summarisation as the most common use cases. Transformer-based architectures, particularly BioBERT and ClinicalBERT, achieved state-of-the-art performance. DocMate's RAG-based architecture addresses the retraining challenge by allowing knowledge base updates without full model retraining.

2.1.6 Retrieval-Augmented Generation for Medical QA (arXiv, 2023)

This paper proposed a RAG pipeline for medical QA achieving accuracy improvements of 12–18% compared to pure generative models. The RAG approach reduced hallucination rates significantly by grounding responses in retrieved medical documents. This architecture forms the basis of DocMate's AI chatbot pipeline.

2.1.7 Symptom Checker AI with Decision Trees (Nature Digital Medicine, 2021)

This study compared AI-based symptom checkers to physician triage for 1,669 clinical vignettes. The study emphasised that AI symptom checkers' primary value lies in improving

pre-visit triage efficiency and reducing inappropriate emergency department visits. DocMate positions its AI chatbot as a triage and recommendation tool rather than a diagnostic system.

2.1.8 Mobile Health Medication Reminders (JMIR mHealth, 2022)

This randomised controlled trial evaluated smartphone-based medication reminder systems, demonstrating a 27% improvement in medication adherence. DocMate incorporates these findings into its medication reminder module, enabling reminders to be automatically generated from doctor-issued prescriptions.

2.1.9 Telepsychiatry Platform with Sentiment Analysis (Springer, 2023)

This paper described a telepsychiatry platform incorporating real-time sentiment analysis, reporting video consultation latency below 400ms. DocMate draws inspiration from this platform's real-time AI assistance model in its emergency detection layer.

2.1.10 Comparison Table

Platform / Study	AI Component	Scheduling	Video	Med Records	Drug Info	Alerts	
NLP Chatbots (IEEE, 2022)	NLP/BERT	No	No	No	No	No	
Hospital Mgmt (ACM, 2021)	Rule-Based	Yes	No	Partial	No	No	
MedAssist (JMIR, 2021)	No	Limited	No	Yes	No	No	
WebRTC Telemedicine (IEEE, 2020)	No	No	Yes	No	No	No	
RAG Medical QA (arXiv, 2023)	RAG/LLM	No	No	No	No	No	
Symptom Checker (Nature, 2021)	ML/Decision Tree	No	No	No	No	No	
mHealth Reminder (JMIR, 2022)	Rule-Based	No	No	Partial	No	No	
DocMate (Proposed)	NLP + RAG + ML	Yes	Yes	Yes	Yes	Yes	

Table 2.1: Comparison of Related Works

2.1.11 Research Gap Identification

The reviewed literature highlights significant progress in isolated healthcare domains such as chatbot-based symptom assessment, web-based appointment systems, telemedicine platforms, and electronic medical record management. However, a clear research and implementation gap remains in the integration of these capabilities into a single intelligent healthcare ecosystem. Most existing systems address only one or two aspects of the patient journey and therefore fail to provide a seamless end-to-end experience.

The major gaps identified from the literature are as follows:

- Existing healthcare chatbots often provide symptom-based responses but do not directly connect users with suitable specialists or appointment booking workflows.
- Appointment management systems are generally rule-based and lack AI-assisted triage capabilities.
- Telemedicine platforms enable video consultation but frequently do not integrate electronic health records, medicine guidance, or emergency escalation mechanisms.
- Medical question-answering systems based on large language models may suffer from hallucination unless grounded through retrieval-based approaches.
- Very few systems are designed specifically with the socio-technical realities of developing countries in mind, including limited specialist access, bilingual symptom reporting, and fragmented healthcare data practices.

DocMate is designed to address this gap by integrating AI-powered symptom analysis, doctor recommendation, appointment scheduling, real-time consultation, health record management, medicine information, and emergency alert functionality into one unified web platform.

2.1.12 Justification of Proposed System

The proposed system is justified on both practical and technical grounds. Practically, patients require a convenient digital platform that reduces the complexity of searching for doctors, booking appointments, and accessing healthcare guidance. Technically, recent advances in NLP, semantic retrieval, and browser-based real-time communication make it feasible to build a smart, integrated healthcare system with relatively low infrastructure overhead.

The use of Retrieval-Augmented Generation is particularly justified in the healthcare domain because it improves trustworthiness by grounding responses in curated medical knowledge instead of relying solely on generative output. Similarly, WebRTC is selected because it allows secure, low-latency audio-video communication directly in the browser without requiring additional software installation. Role-based access control and encrypted data management further make the system appropriate for healthcare use cases where confidentiality and controlled access are essential.

2.2 Stakeholders List

Stakeholder	Role	Primary Interests	Interaction Level
Patients	Primary end users	Appointment booking, AI guidance, health records	High – daily
Doctors	Healthcare providers	Manage appointments, patient data, consultations	High – professional
System Administrator	Platform manager	User management, analytics, configuration	Medium – administrative
Developers (FYP Team)	System builders	Deliver functional, tested, documented system	High – design & implementation
Project Supervisor	Academic guide	Technical correctness, research quality	Medium – review
Head of Department	Institutional oversight	Academic standards, project completion	Low – approval

Emergency Contacts	Notified in alerts	Receive emergency notifications	Low – notified when triggered
--------------------	--------------------	---------------------------------	-------------------------------

Table 2.2: Stakeholders Analysis

2.3 Requirements Elicitation

2.3.1 Functional Requirements

FR01 – Patient Registration	
Req. No.	Requirement
FR01-01	Allow new patients to register by providing name, email, date of birth, phone, and password.
FR01-02	Validate all input fields ensuring email uniqueness, password strength, and valid phone format.
FR01-03	Send a verification email with a unique time-limited confirmation link upon successful registration.
FR01-04	Allow patients to upload a profile picture and optional health profile information.
FR01-05	Provide inline error messages identifying the specific validation failure for each field.

FR02 – User Authentication	
Req. No.	Requirement
FR02-01	Allow registered users to log in using their verified email and password.
FR02-02	Authenticate users via Firebase Authentication with role-based access (Patient, Doctor, Administrator).
FR02-03	Display generic error messages for incorrect credentials to prevent credential enumeration.
FR02-04	Provide a password reset flow via email OTP.
FR02-05	Support session management with configurable token expiry and automatic inactivity logout.
FR02-06	Implement JWT-based API authentication for all protected backend endpoints.

FR03 – Appointment Management	
Req. No.	Requirement
FR03-01	Allow patients to search for doctors by medical specialty, name, or location.
FR03-02	Display doctor profiles including qualifications, specialty, experience, ratings, and available slots.
FR03-03	Allow patients to select a time slot and confirm a booking with optimistic slot locking.

FR03-04	Send booking confirmation notifications via email and in-app notification within 60 seconds.
FR03-05	Send automated reminder notifications 24 hours and 1 hour before appointments.
FR03-06	Allow patients to reschedule or cancel appointments up to 2 hours before the appointment time.
FR03-07	Allow doctors to define their availability and manage schedules through the doctor portal.
FR03-08	Prevent double-booking using MySQL row-level locking (SELECT FOR UPDATE).

FR04 – AI Chatbot

Req. No.	Requirement
FR04-01	Provide an AI-powered chatbot accessible to logged-in patients from the dashboard.
FR04-02	Use NLP to understand and interpret free-text patient symptom descriptions with medical entity extraction.
FR04-03	Recommend appropriate medical specialists based on symptom analysis with a confidence threshold.
FR04-04	Use RAG to retrieve and present relevant information from the curated medical knowledge base.
FR04-05	Handle general healthcare FAQs including medication queries, preventive care, and emergency guidance.
FR04-06	Maintain conversation context across multiple turns within a single session.
FR04-07	Provide an option to book an appointment with the recommended specialist from within the chatbot.
FR04-08	Fall back to recommending a GP when symptom descriptions are ambiguous.

FR05 – Video Consultation

Req. No.	Requirement
FR05-01	Allow patients and doctors to initiate real-time video consultation at the scheduled appointment time.
FR05-02	Implement peer-to-peer audio-video communication using WebRTC with STUN/TURN fallback.
FR05-03	Provide an in-consultation text chat window for sharing information and documents.
FR05-04	Allow doctors to share their screen during consultations.
FR05-05	Generate and store a session summary record upon consultation completion.
FR05-06	Provide a virtual waiting room interface for the patient before the doctor joins.

FR06 – Medical Records

Req. No.	Requirement
FR06-01	Allow patients to view their complete health history including consultations, diagnoses, and prescriptions.
FR06-02	Allow patients to upload health documents (PDF or image, max 10 MB).
FR06-03	Allow doctors to create and update consultation notes, diagnoses, and prescriptions.
FR06-04	Implement RBAC so patients see their own records and doctors see only their patients' records.
FR06-05	Provide AES-256 encrypted storage for all health documents and records.
FR06-06	Allow doctors and patients to download consultation records in PDF format.

FR07 – Medicine Information

Req. No.	Requirement
FR07-01	Provide a searchable medicine database accessible to both patients and doctors.
FR07-02	Display medicine details including generic name, brand names, indications, dosage, and side effects.
FR07-03	Allow the administrator to add, update, or remove medicine entries via the admin dashboard.
FR07-04	Integrate medicine lookup functionality within the chatbot for contextual drug queries.

FR08 – Emergency Alerts

Req. No.	Requirement
FR08-01	Continuously monitor chatbot symptom inputs for patterns indicative of medical emergencies.
FR08-02	Trigger an emergency alert UI overlay within 2 seconds of detecting emergency-level symptoms.
FR08-03	Automatically notify registered emergency contacts via email/SMS within 60 seconds.
FR08-04	Notify available on-duty physicians registered on the platform.
FR08-05	Log all emergency events with timestamp, symptom description, and resolution status.
FR08-06	Provide configurable medication reminder notifications at patient-specified times.

FR09 – Doctor Portal

Req. No.	Requirement
FR09-01	Provide doctors with a dashboard displaying today's appointments, schedule, and notifications.
FR09-02	Allow doctors to view comprehensive patient profiles and medical histories during consultations.
FR09-03	Allow doctors to write digital prescriptions with medication name, dosage, frequency, and duration.
FR09-04	Allow doctors to update consultation notes and diagnostic summaries with auto-save.
FR09-05	Allow doctors to set and modify their availability calendar with recurring weekly schedule support.
FR09-06	Allow doctors to initiate or join video consultation sessions from the appointment dashboard.

FR10 – Administrator Dashboard

Req. No.	Requirement
FR10-01	Provide the administrator with a dedicated dashboard with real-time platform statistics.
FR10-02	Allow the administrator to manage patient and doctor accounts with audit logging.
FR10-03	Allow the administrator to review and respond to reported issues from a unified view.
FR10-04	Provide analytics on appointment volumes, chatbot interactions, and system performance.
FR10-05	Allow the administrator to configure platform settings including specialties and working hours.

FR11 – User Logout

Req. No.	Requirement
FR11-01	Allow all user types to securely log out from any active session.
FR11-02	Invalidate all active session tokens and Firebase authentication tokens upon logout.
FR11-03	Redirect the user to the public login page upon successful logout.
FR11-04	Implement automatic logout after a configurable period of inactivity.

2.3.2 Non-Functional Requirements

NFR01 – Performance

Req. No.	Requirement
NFR01-01	All standard page loads must complete within 3 seconds on a broadband connection (≥ 10 Mbps).
NFR01-02	The system must support a minimum of 50 concurrent users without performance degradation above 1% error rate.

NFR01-03	Real-time video consultation latency must remain below 500ms under normal network conditions.
NFR01-04	Chatbot responses must be generated and displayed within 3 seconds of user input submission.
NFR01-05	Database query response times must remain below 500ms for standard CRUD operations.

NFR02 – Security

Req. No.	Requirement
NFR02-01	All data transmission must occur over HTTPS using TLS 1.2 or higher.
NFR02-02	Patient health data must be encrypted at rest using AES-256 encryption.
NFR02-03	Role-based access control must prevent users from accessing resources outside their scope.
NFR02-04	Passwords must be hashed using bcrypt with a minimum work factor of 12.
NFR02-05	The system must implement CORS policies restricting API access to authorised domains only.
NFR02-06	All API endpoints must validate and sanitise inputs to prevent SQL injection, XSS, and CSRF attacks.

NFR03 – Usability

Req. No.	Requirement
NFR03-01	A new user must be able to complete registration and book their first appointment within 5 minutes.
NFR03-02	All primary tasks must be accessible within 3 clicks from the dashboard.
NFR03-03	All form validations must display inline error messages in plain language.
NFR03-04	The platform must provide adequate colour contrast ratios meeting WCAG 2.1 Level AA standards.

NFR04 – Compatibility

Req. No.	Requirement
NFR04-01	The application must function correctly on the two most recent major versions of Chrome, Firefox, Safari, and Edge.
NFR04-02	The responsive design must display correctly on screen widths from 360px (mobile) to 1920px (desktop).
NFR04-03	WebRTC video consultation must be supported on all browsers where WebRTC is natively available.

2.3.3 Requirements Traceability Matrix

Req. ID	Description	Use Case	System Diagram	Test Case
FR01	Patient Registration	UC-01	SD-01	TC-01
FR02	User Authentication	UC-02	SD-01	TC-02
FR03	Appointment Management	UC-03	SD-02	TC-03
FR04	AI Chatbot	UC-04	SD-03	TC-04
FR05	Video Consultation	UC-05	SD-04	TC-05
FR06	Medical Records	UC-06	SD-05	TC-06
FR07	Medicine Information	UC-07	SD-05	TC-07
FR08	Emergency Alerts	UC-08	SD-06	TC-08
FR09	Doctor Portal	UC-09	SD-07	TC-09
FR10	Admin Dashboard	UC-10	SD-08	TC-10
FR11	User Logout	UC-11	SD-01	TC-11
NFR01	Performance	—	Architecture	Performance Tests
NFR02	Security	—	SD-01	Security Tests
NFR03	Usability	—	—	UAT
NFR04	Compatibility	—	—	Compatibility Tests

Table 2.18: Requirements Traceability Matrix

2.4 Feasibility Analysis

2.4.1 Technical Feasibility

The proposed system is technically feasible because the selected technologies are mature, well-documented, and widely used in modern web application development. React.js provides a robust frontend framework for building responsive and interactive user interfaces, while Flask offers flexibility for backend API development and AI pipeline integration. MySQL is suitable for structured relational data such as users, appointments, prescriptions, and records. The use of Firebase Authentication simplifies secure identity management, and WebRTC supports real-time consultations without requiring dedicated video conferencing software.

The AI components are also technically feasible. NLP-based symptom extraction, FAISS-based retrieval, and lightweight LLM deployment can be implemented within the computational constraints of an academic project by using optimised models and curated datasets.

2.4.2 Economic Feasibility

DocMate is economically feasible for an academic final year project because it relies largely on open-source tools and free-tier cloud resources. Technologies such as React, Flask,

MySQL, FAISS, spaCy, and Redis are free to use. Limited usage tiers of Firebase, Twilio, and SendGrid can support prototype-level deployment and testing. Since the project does not require expensive proprietary healthcare infrastructure, the development cost remains within a manageable student budget.

2.4.3 Operational Feasibility

The system is operationally feasible because its intended users are already familiar with digital communication platforms such as mobile apps, websites, and messaging systems. Patients can interact with the platform using standard web browsers, while doctors can manage schedules and consultations through a dedicated portal. The interfaces are designed to reduce complexity and support intuitive usage. Since the system supplements rather than replaces doctors, it aligns well with existing healthcare workflows.

2.4.4 Schedule Feasibility

The project is feasible within the academic timeline due to modular development using Agile-Scrum. Core modules such as authentication, scheduling, chatbot integration, records management, and video consultation can be developed incrementally across planned sprints. The division of work among team members also supports timely completion. Although some advanced features may require optimisation in later stages, the core deliverables remain realistic for the available development period.

2.5 Research Methodology

2.5.1 Research Nature

The DocMate project follows an applied research approach because it is aimed at solving a real-world problem through the design and development of a practical software solution. Unlike purely theoretical studies, applied research focuses on using technical knowledge, existing frameworks, and system design principles to address identifiable issues in a target domain. In this case, the target domain is digital healthcare service delivery, particularly the problems associated with appointment management, specialist discovery, patient-doctor communication, and record accessibility.

The project also has a development-oriented research character, as it involves both analytical and constructive phases. The analytical phase includes problem identification, literature review, requirement elicitation, and feasibility assessment, while the constructive phase focuses on system design, implementation planning, testing, and evaluation.

2.5.2 Research Approach

The research approach adopted for this project is primarily qualitative and design-based, supported by technical analysis and system modelling. The project begins with an investigation of existing healthcare system limitations, followed by the review of prior studies and platforms in telemedicine, healthcare chatbots, electronic health records, and appointment management systems. Based on the identified gaps, a proposed solution is conceptualised and modelled through diagrams, requirements, workflows, and architecture.

This approach is appropriate because the project is not limited to numerical hypothesis testing; instead, it focuses on understanding the healthcare workflow, identifying pain points, and creating an integrated digital platform that addresses those challenges. Technical validation is then achieved through system testing, module integration, and planned performance evaluation.

2.5.3 Requirement Gathering Method

The requirements for DocMate were gathered through a combination of the following methods:

a) Literature-Based Analysis

Published research articles, healthcare technology studies, and telemedicine literature were reviewed to understand common system features, implementation strategies, and technical challenges in digital healthcare systems.

b) Problem Domain Observation

The team examined the practical issues commonly experienced in local healthcare settings, such as long patient queues, appointment delays, poor record maintenance, and limited access to specialist consultation. These observations helped in identifying user pain points relevant to the proposed system.

c) Comparative Feature Analysis

Existing platforms and studies were analysed to determine what functionalities are commonly available and what important gaps remain unaddressed. This process helped define the unique contribution of DocMate, especially in terms of integration and AI-assisted healthcare guidance.

d) Stakeholder-Centred Thinking

The system requirements were structured around the expected needs of primary stakeholders, including patients, doctors, administrators, and emergency contacts. This ensured that the final design considered usability, role-specific workflows, and operational practicality.

2.5.4 System Development Methodology

The development of DocMate follows the Agile-Scrum methodology, which supports iterative and incremental software development. Scrum was selected because it is suitable for projects where requirements evolve gradually and where the system can be built in small functional increments. Since DocMate consists of multiple modules such as authentication, chatbot, booking, records, consultation, and notifications, an iterative approach allows each component to be designed, tested, and improved step by step.

Under this methodology, the project is divided into planned sprints. Each sprint focuses on a specific set of goals and deliverables, such as requirement analysis, system design, AI prototype development, backend implementation, frontend integration, and testing. This method also supports continuous review, early issue detection, and better coordination among team members.

2.5.5 Research and Development Stages

The methodology of the project can be divided into the following stages:

Stage 1: Problem Identification

The first stage involved identifying key shortcomings in current healthcare appointment and consultation workflows. This included delays in booking, poor access to specialists, lack of integrated records, and absence of intelligent triage support.

Stage 2: Literature Review and Gap Analysis

In the second stage, relevant studies and existing systems were reviewed to understand the current state of healthcare technologies and identify missing features or integration gaps.

Stage 3: Requirement Engineering

Functional and non-functional requirements were elicited, documented, and organised according to user roles and system expectations.

Stage 4: System Design

The conceptual architecture, use case diagrams, activity diagrams, sequence diagrams, class diagram, and ER diagram were prepared to provide a complete model of the proposed system.

Stage 5: Prototype Planning and Implementation

The technologies and tools for implementation were selected, and the project modules were planned according to Agile development cycles. Initial work was begun on the backend, frontend, and AI pipeline.

Stage 6: Testing and Validation

A structured testing strategy was designed to evaluate correctness, security, usability, and performance of the system modules. This includes unit testing, integration testing, system testing, and user acceptance testing.

2.5.6 Validation Strategy

The proposed system is validated through both technical validation and functional validation. Technical validation focuses on whether the selected technologies, architecture, and modules can perform the required tasks under expected conditions. Functional validation checks whether the implemented features align with the stated requirements and user workflows.

Validation methods include:

- test case execution for major modules
- integration testing of connected workflows
- usability-oriented acceptance scenarios
- performance-related checks for response time and concurrent use

- security-oriented validation for authentication, access control, and input handling

These methods help ensure that the system is not only conceptually sound but also practically usable and reliable.

2.5.7 Ethical and Practical Boundaries of the Methodology

Since the system operates in the healthcare domain, the research methodology recognises important ethical and practical boundaries. The chatbot is positioned as an assistive guidance tool and not as a diagnostic engine. The project does not claim clinical certification, and all outputs are framed within the boundaries of academic prototyping and responsible software development. Similarly, privacy, user consent, and secure access are treated as fundamental design concerns throughout the methodology.

2.5.8 Summary

The research methodology adopted for DocMate combines applied research, system analysis, stakeholder-oriented requirement engineering, and iterative Agile development. This methodology provides a structured pathway from problem identification to design, implementation, testing, and evaluation. It is well suited to a healthcare technology project that aims to produce both academic insight and a practical software prototype.

Chapter 3 – System Design

3.1 Use Case Diagrams

3.1.1 Patient Use Case Diagram

The patient use case diagram captures all interactions available to a registered patient on the DocMate platform. The primary actor is the Patient, who interacts through seven main use cases: Register/Login, AI Symptom Check, Book Appointment, Attend Video Consultation, View/Upload Medical Records, Search Medicine Information, and Manage Profile.

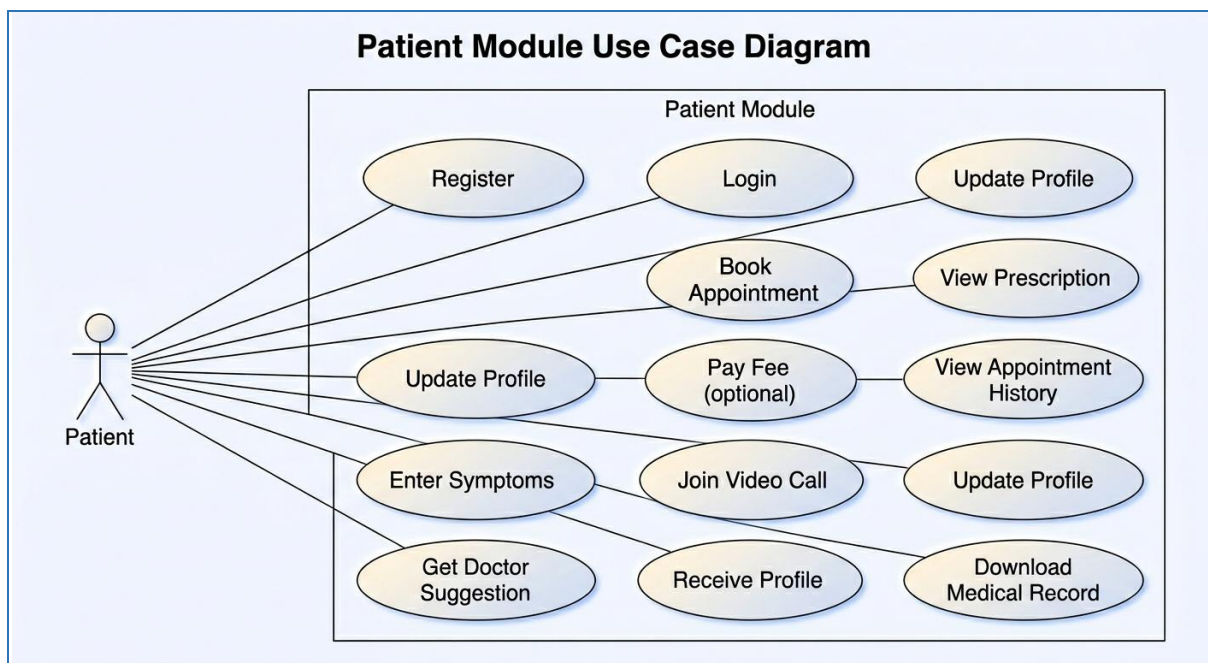


Figure 3.1: Use Case Diagram – Patient

3.1.2 Doctor Use Case Diagram

The doctor use case diagram illustrates all interactions available to a verified doctor. Use cases include Login/Profile Management, Manage Appointment Schedule, Conduct Video Consultation, Update Patient Records, View Patient History, Write Prescriptions, and Receive Notifications.

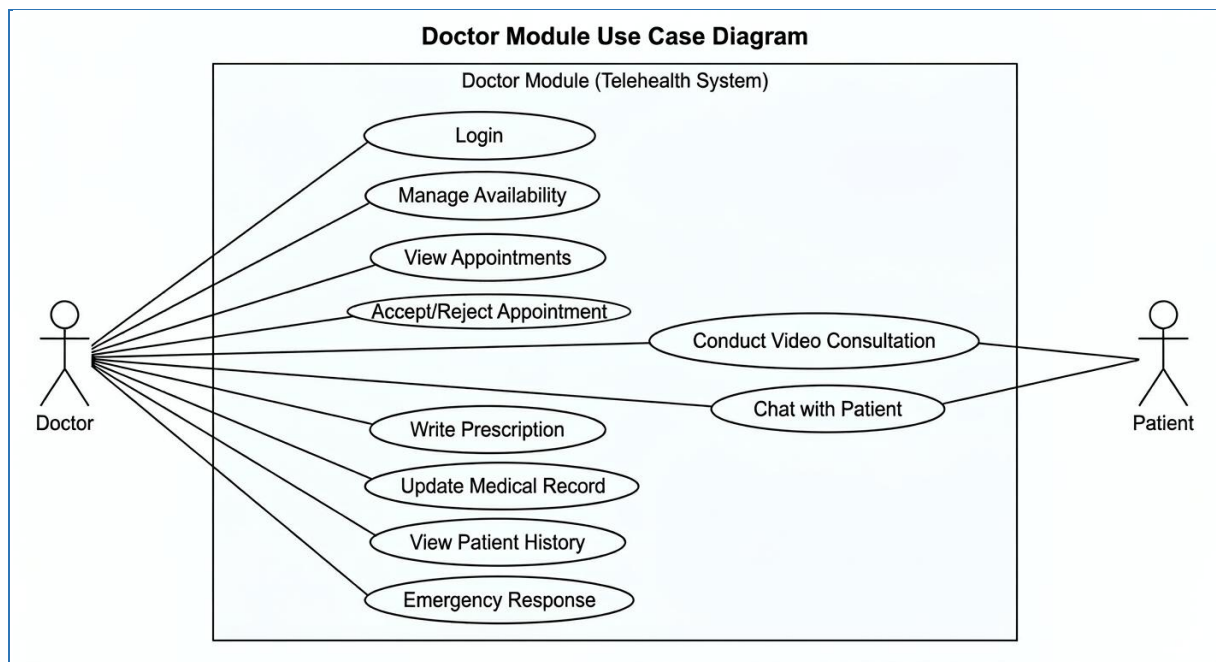


Figure 3.2: Use Case Diagram – Doctor

3.1.3 Administrator Use Case Diagram

The administrator use case diagram captures platform management tasks including Manage Users, View Analytics, Configure Platform, Manage Medicine Database, and Review Audit Logs.

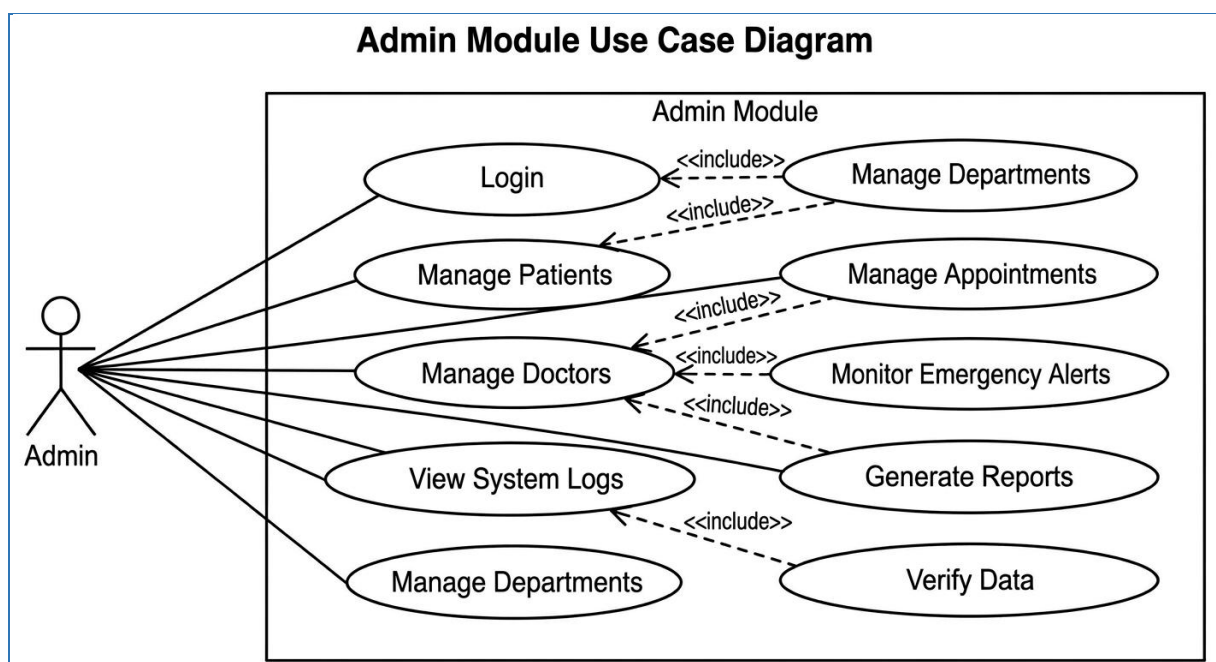


Figure 3.3: Use Case Diagram – Admin

3.2 Activity Diagrams

3.2.1 Patient Registration Activity

The registration activity flow begins when a new user navigates to the registration page. The system performs three parallel validation checks: email uniqueness, input format, and phone number format. Upon successful validation, the system creates a user record with 'pending'

status, hashes the password using bcrypt, and dispatches a verification email via SendGrid with a 24-hour confirmation link.

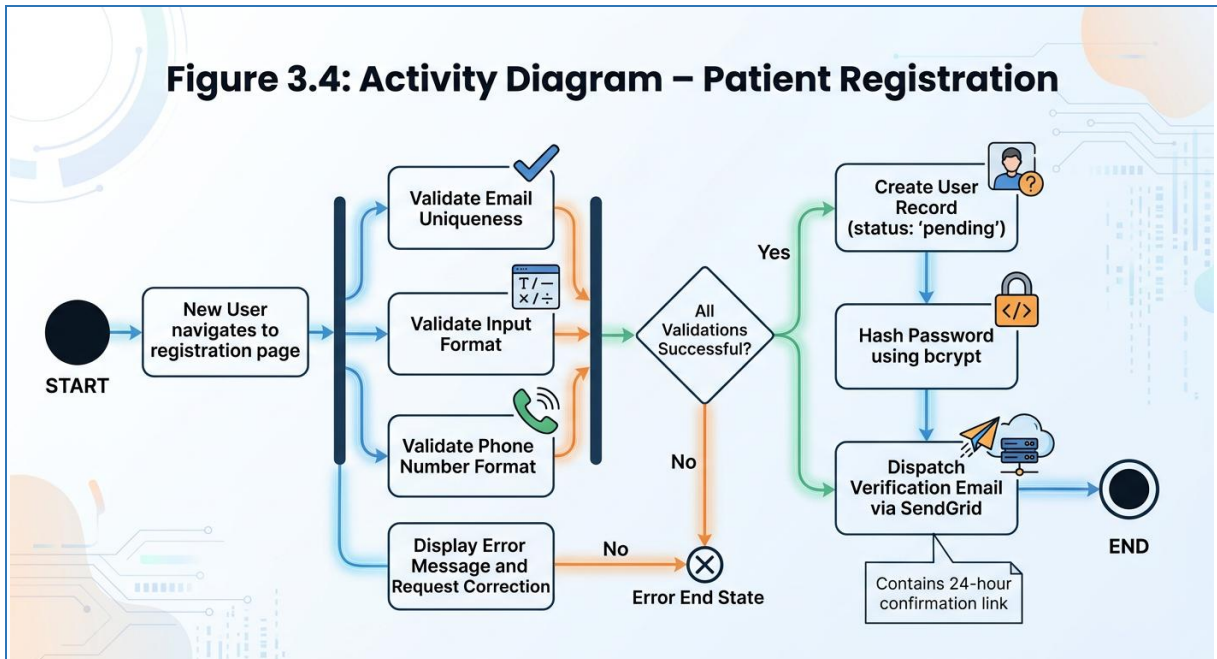


Figure 3.4: Activity Diagram – Patient Registration

3.2.2 Appointment Booking Activity

The appointment booking flow begins when a logged-in patient navigates to the booking section. The system dynamically generates available slots and implements a 3-minute optimistic lock on the selected slot. Upon confirmation, the appointment is created and asynchronous notifications are dispatched via the Celery task queue.

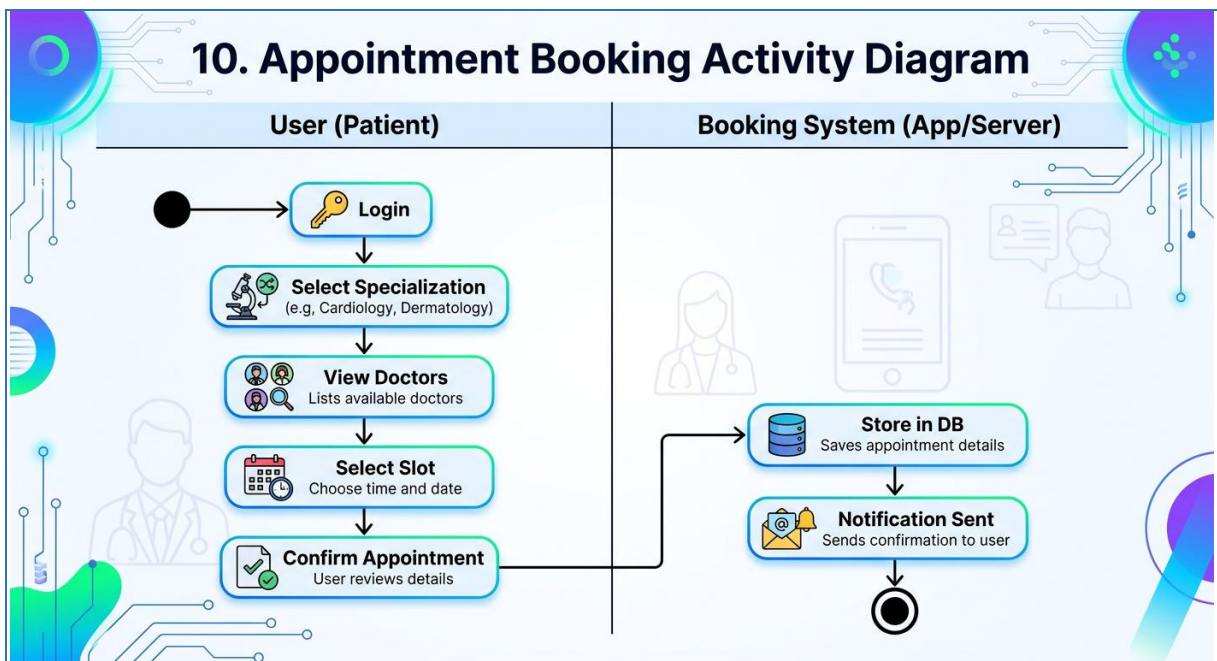


Figure 3.5: Activity Diagram – Appointment Booking

3.2.3 AI Symptom Analysis Activity

Patient input is pre-processed through tokenisation, stop word removal, and medical entity extraction. The RAG pipeline retrieves top-K semantically similar document chunks. In parallel, the emergency keyword classifier evaluates input with a 0.85 similarity threshold. The LLaMA 3 model generates a contextualised response with specialist recommendation.

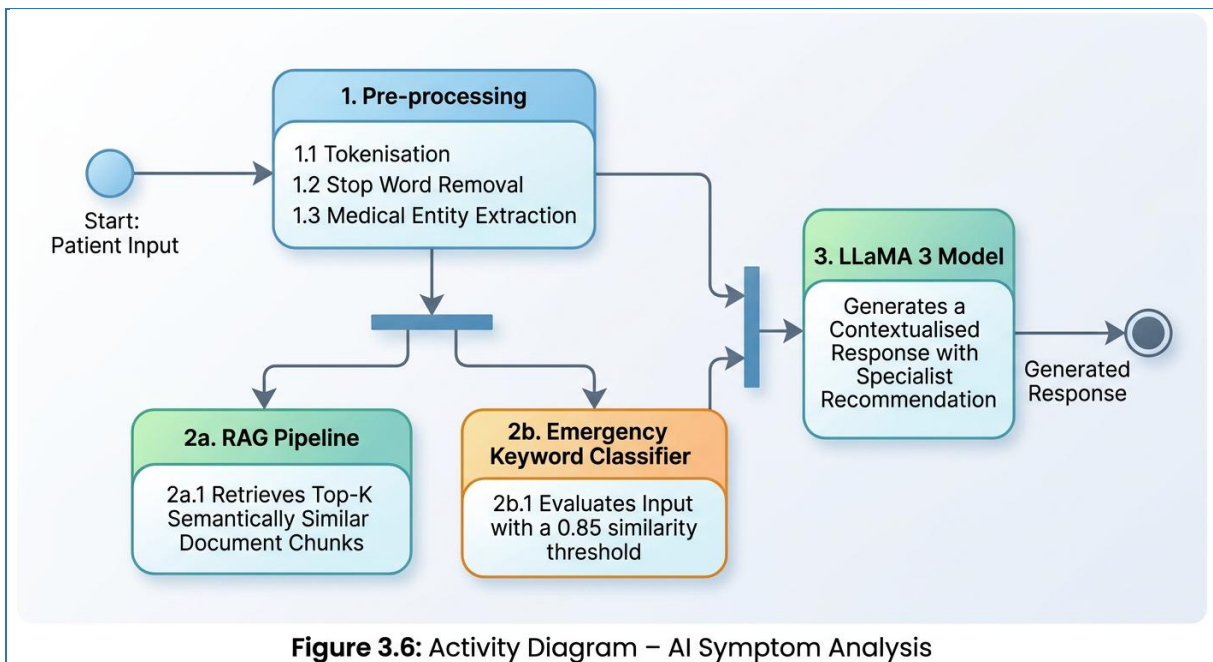


Figure 3.6: Activity Diagram – AI Symptom Analysis

3.2.4 Video Consultation Activity

The video consultation flow begins at the scheduled appointment time. The WebRTC signalling handshake is initiated via the Flask-SocketIO server. ICE candidates are exchanged and a peer-to-peer connection is established, falling back to Twilio TURN servers if STUN-based NAT traversal fails within 5 seconds.

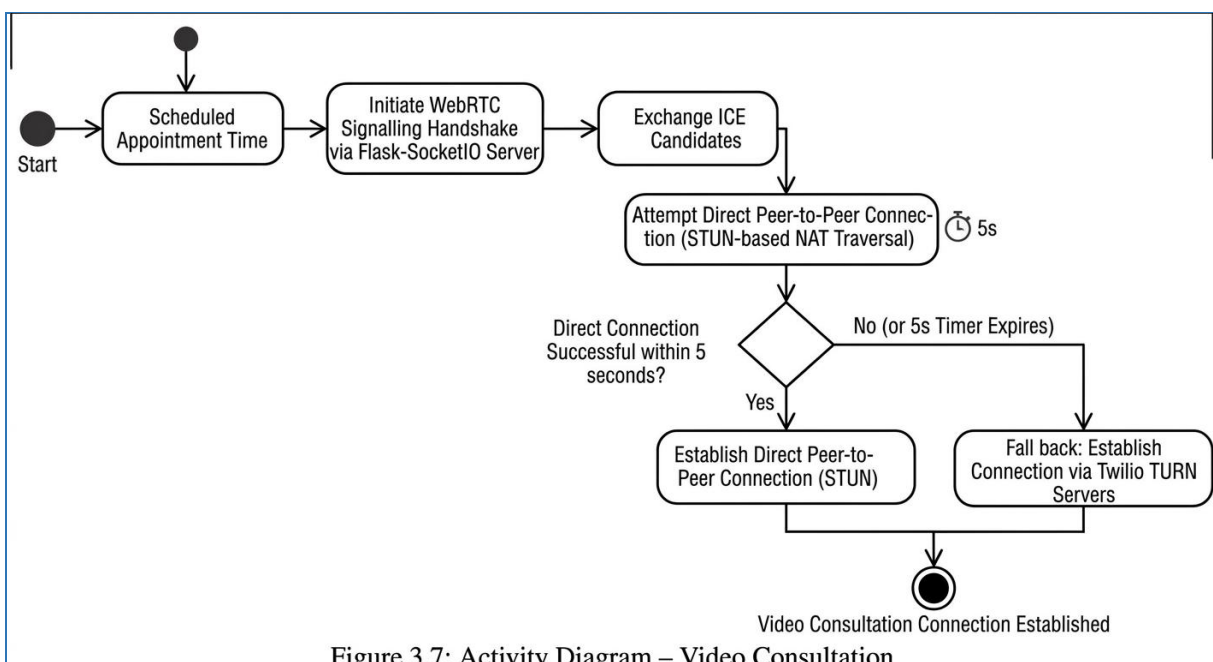


Figure 3.7: Activity Diagram – Video Consultation

3.2.5 Medical Records Activity

The medical records activity supports two parallel workflows: patient-side record viewing and document upload (to AWS S3 via pre-signed URLs), and doctor-side record creation and update during or after consultation. Role ownership is validated before any record access.

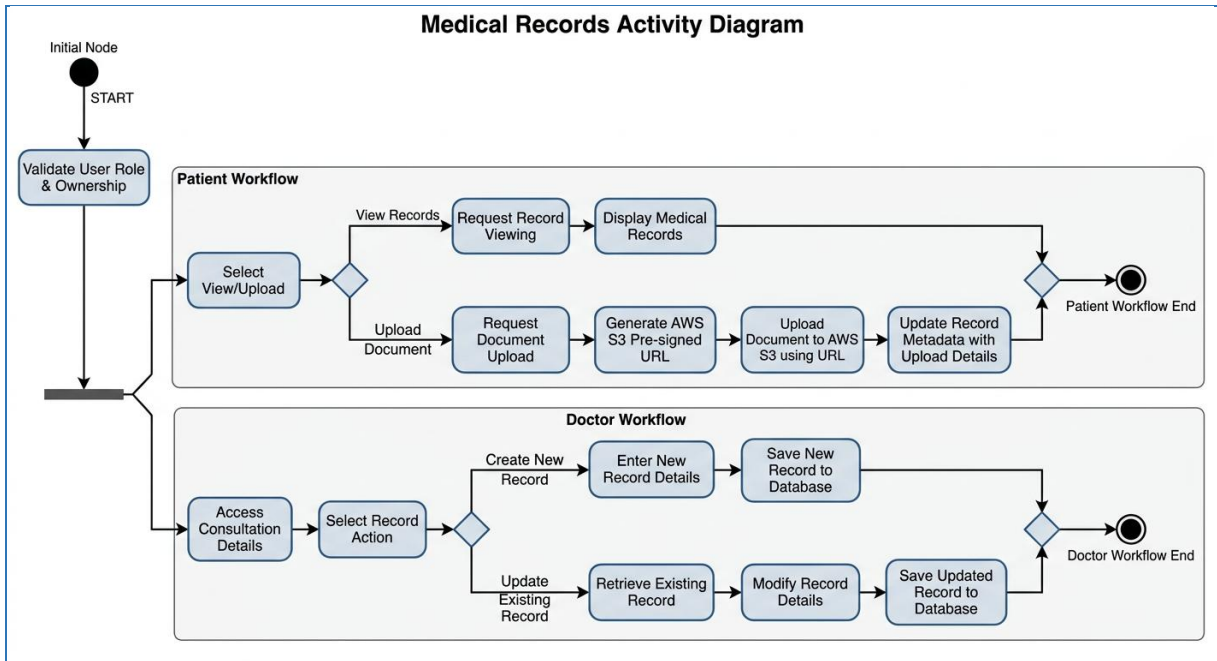


Figure 3.8: Activity Diagram – Medical Records

3.3 Sequence Diagrams

3.3.1 Appointment Booking Sequence

Participants: Patient (Browser), React Frontend, Flask API, MySQL Database, Email Service. The patient submits filters, receives doctor availability, selects a slot, and the Flask API uses SELECT FOR UPDATE to lock the slot atomically before creating the appointment record and dispatching confirmation emails asynchronously via Celery.

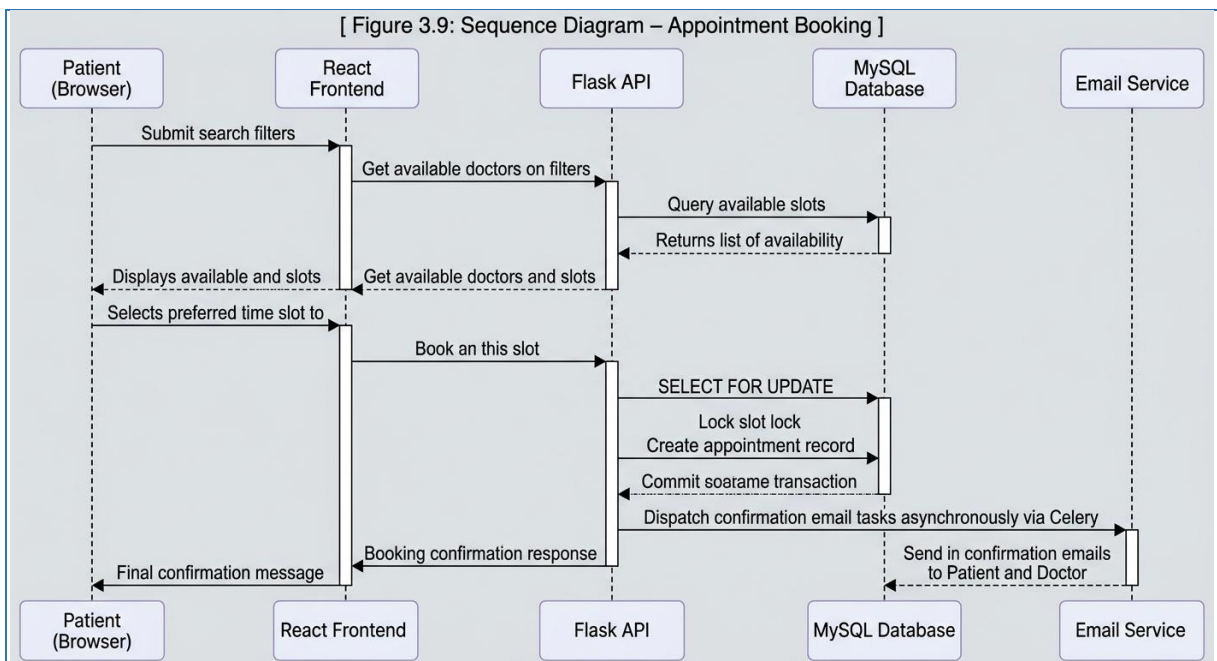


Figure 3.9: Sequence Diagram – Appointment Booking

3.3.2 Authentication Sequence

Participants: User (Browser), React Frontend, Firebase Auth, Flask API, MySQL Database. Firebase Authentication verifies credentials and issues a Firebase ID Token. The Flask backend validates the token via Firebase Admin SDK, retrieves the user role from MySQL, and issues an application-level JWT with role claims.

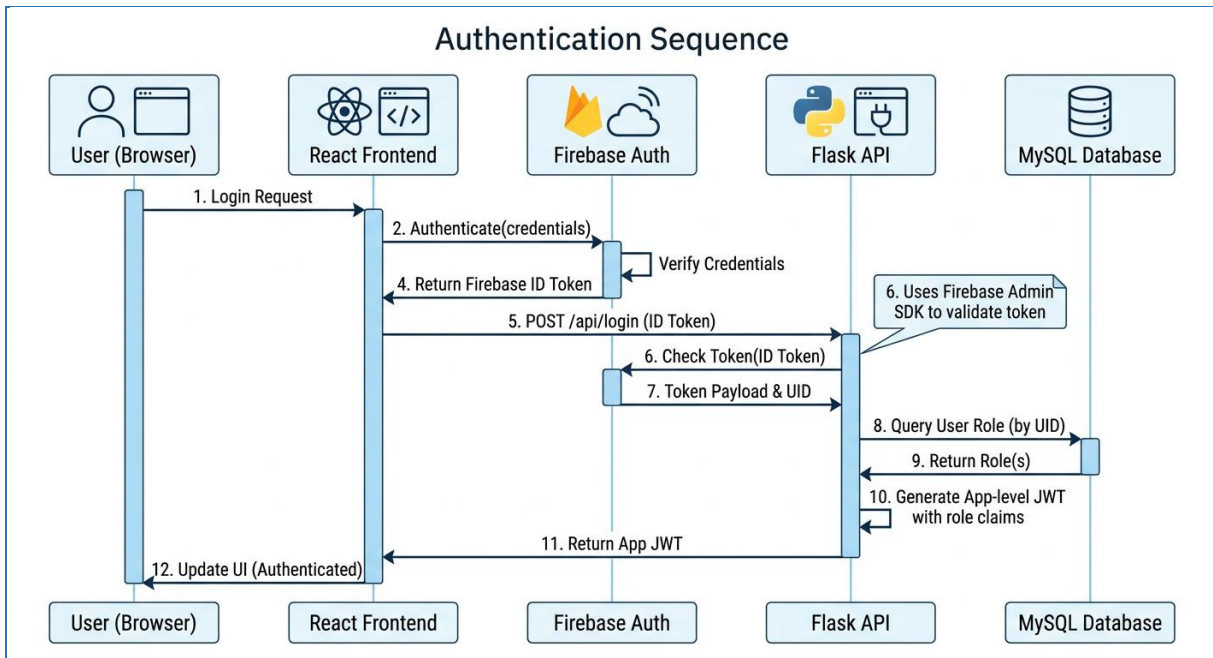


Figure 3.10: Sequence Diagram – Authentication

3.3.3 AI Chatbot Sequence

Participants: Patient (Browser), React Frontend, Flask API, NLP Engine, RAG Pipeline, LLM Service, MySQL Database. Patient input is tokenised, medical entities extracted, and the FAISS-indexed knowledge base is queried for top-5 document chunks. The LLaMA 3 model generates a response (≤ 200 words) which is parsed for specialist recommendations.

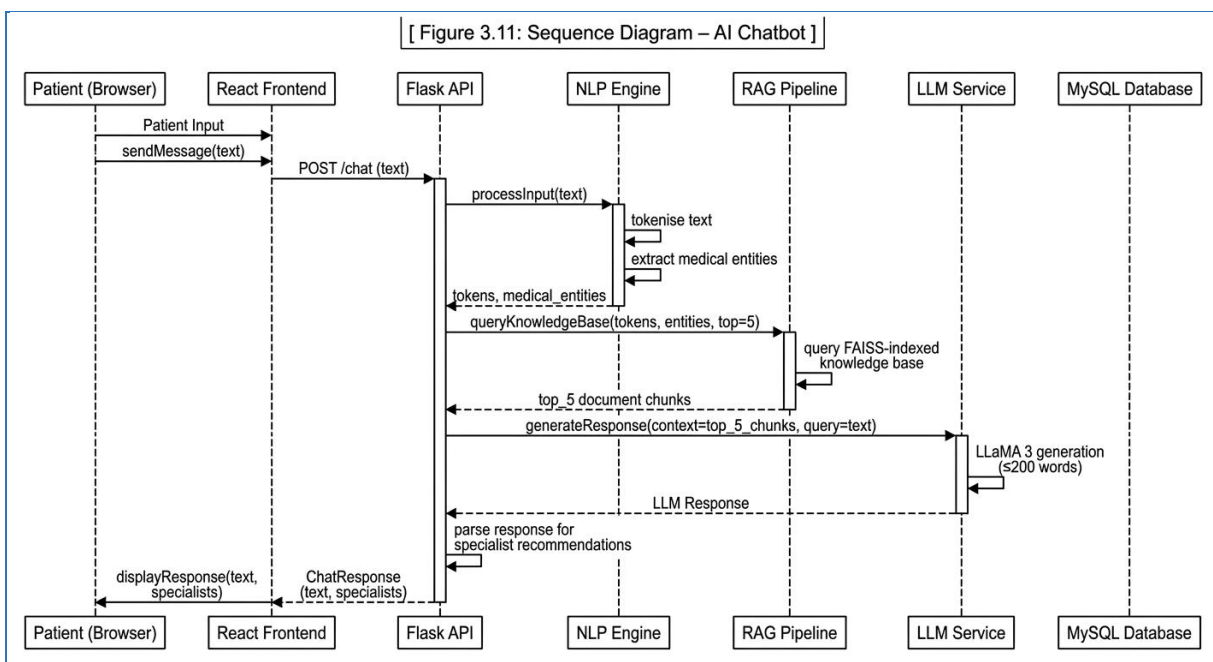


Figure 3.11: Sequence Diagram – AI Chatbot

3.3.4 Video Consultation Sequence

Participants: Patient (Browser), Doctor (Browser), Flask-SocketIO Server, Twilio TURN Server. SDP offer/answer and ICE candidates are exchanged via the signalling server. If direct P2P connection fails within 5 seconds, Twilio TURN credentials are issued for relay.

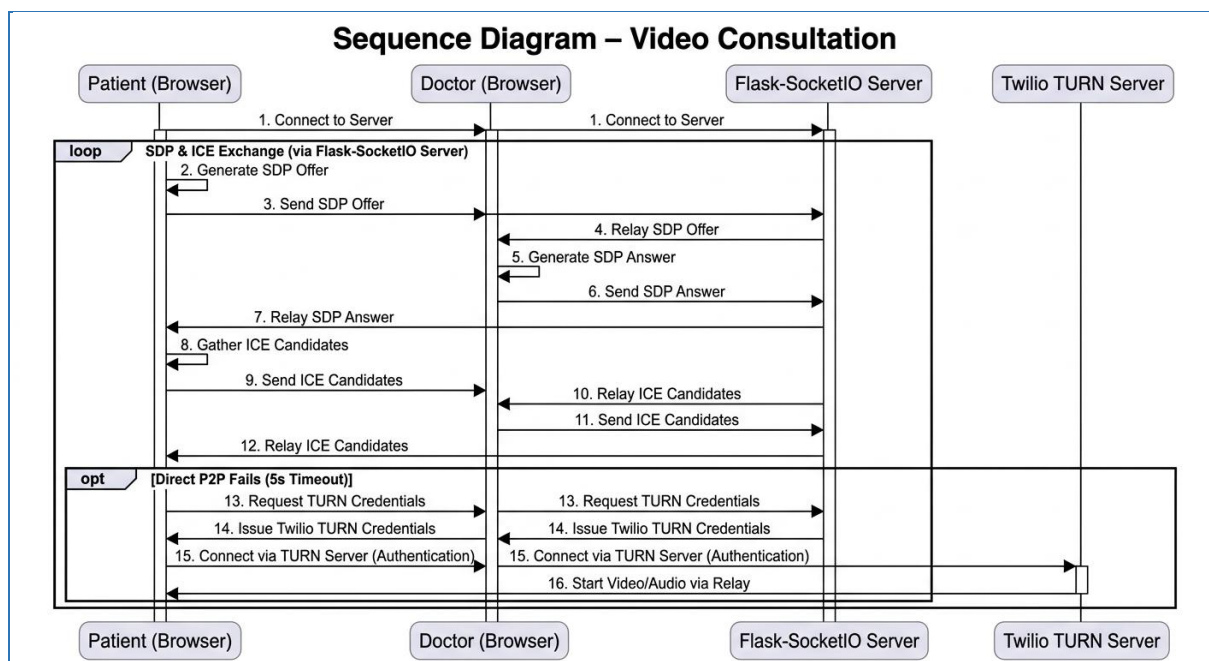


Figure 3.12: Sequence Diagram – Video Consultation

3.4 Software Architecture

3.4.1 Architecture Overview

DocMate follows a client-server architecture with three primary tiers: Presentation Tier (React.js frontend), Application Tier (Flask REST API and AI services), and Data Tier (MySQL and Firebase). A fourth supporting tier comprises external services including Firebase Authentication, WebRTC/Twilio, and Email/SMS providers.

Tier	Technology	Responsibilities
Presentation	React.js, Tailwind CSS	Renders UI for all three portals; manages client-side state.
Application	Flask (Python), Gunicorn, NLP Engine, RAG Pipeline	Processes business logic; exposes RESTful API; runs AI pipeline.
Data	MySQL 8.0	Stores users, appointments, health records, prescriptions, medicines.
Real-Time	Firebase, WebRTC/Socket.IO	Authentication state, in-app notifications, video signalling.
External Services	Twilio, SendGrid, Firebase Auth	Video relay, transactional email, authentication services.

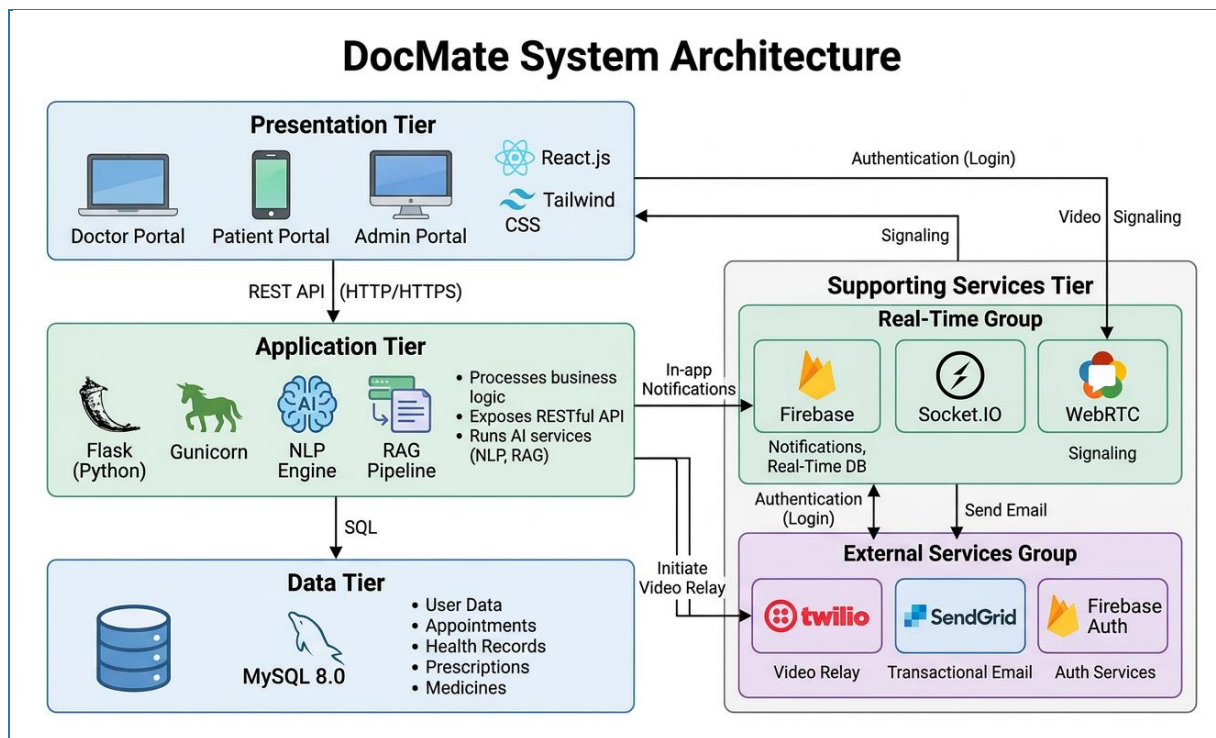


Figure 3.13: Software Architecture Diagram

3.4.2 Architectural Design Principles

The architecture of DocMate is guided by several design principles to ensure maintainability, scalability, security, and usability.

- **Modularity:** Each core feature such as authentication, chatbot, appointments, consultation, and records management is developed as a separate logical module.
- **Separation of Concerns:** The frontend handles presentation logic, the backend handles business rules and APIs, and the data layer manages persistence and retrieval.
- **Security by Design:** Sensitive healthcare data is protected through encryption, token-based authentication, and role-based access control.
- **Extensibility:** The system is designed so that future integrations such as mobile applications, wearable devices, payment systems, or multilingual support can be added with minimal restructuring.
- **Performance Awareness:** The architecture uses asynchronous processing, caching opportunities, and optimised retrieval mechanisms to support fast response times.
- **User-Centred Design:** Interfaces are planned according to the specific needs of patients, doctors, and administrators.

These principles collectively ensure that the system remains practical not only as an academic project but also as a foundation for future real-world deployment.

3.5 UI/UX Design Considerations

3.5.1 Importance of User Interface Design in Healthcare Systems

User interface and user experience design are especially important in healthcare applications because users may interact with the system under stress, urgency, or limited technical familiarity. A poorly designed healthcare platform can create confusion, delay critical actions, and reduce trust in the system. For this reason, the design of DocMate gives special attention to clarity, simplicity, accessibility, and role-specific usability.

The platform is intended for diverse users including patients, doctors, and administrators. These users have different goals, different technical comfort levels, and different workflow expectations. Therefore, the interface is designed to minimise complexity while ensuring that each user can access the most relevant tasks quickly and efficiently.

3.5.2 Design Objectives

The primary UI/UX design objectives of DocMate are as follows:

- To provide a clean and intuitive interface for all user roles
- To reduce the number of steps required for important user actions
- To support fast and error-free appointment booking
- To make symptom entry and chatbot interaction simple and understandable
- To provide visual consistency across patient, doctor, and admin portals
- To ensure responsive behaviour across mobile, tablet, and desktop devices
- To support accessibility through readable typography, colour contrast, and clear feedback

These objectives help ensure that the system remains user-friendly and functionally effective.

3.5.3 Patient-Centred Interface Design

The patient portal is designed to support users who may not have advanced technical knowledge. The interface uses simple labels, familiar navigation patterns, and clearly distinguishable action buttons. Key options such as “Book Appointment,” “Use AI Chatbot,” “View Health Records,” and “Search Medicines” are placed prominently on the dashboard to reduce navigation effort.

The chatbot interface is designed in a conversational and non-intimidating format so that patients can describe symptoms in natural language. Care is taken to ensure that the wording of system responses remains understandable and does not create a false impression of medical diagnosis. Clear call-to-action buttons are included to guide the patient toward the next logical step, such as booking an appointment or reviewing emergency guidance.

3.5.4 Doctor-Focused Workflow Design

The doctor interface is designed to support speed, information visibility, and workflow continuity. Doctors need quick access to appointment schedules, patient histories, consultation notes, and prescription tools. Therefore, the dashboard presents daily appointments, relevant patient summaries, and consultation actions in an organised layout.

The consultation workflow is designed so that doctors can move efficiently between viewing patient records, joining a video session, writing notes, and generating prescriptions.

Information hierarchy is especially important in the doctor portal, as unnecessary clutter can reduce consultation efficiency.

3.5.5 Administrator Interface Design

The administrator dashboard is designed for monitoring and management tasks. This interface prioritises structured data presentation, quick filtering, and overview analytics. Features such as user verification, issue management, specialty configuration, and audit tracking are grouped logically to support administrative efficiency. The admin interface is more data-dense than the patient interface, but it still follows consistency and readability principles.

3.5.6 Consistency and Visual Hierarchy

Consistency is an important design principle throughout the DocMate platform. Similar actions use similar colours, components, and placements across the application. For example, primary action buttons, card layouts, form labels, and status indicators are standardised to reduce cognitive load. This helps users become familiar with the system more quickly.

Visual hierarchy is used to guide attention toward the most important elements on each screen. Headings, buttons, notifications, and emergency-related prompts are visually emphasised using size, spacing, colour, and placement. This becomes particularly important in time-sensitive situations such as emergency symptom detection or imminent appointment reminders.

3.5.7 Responsive Design Strategy

DocMate is intended to be accessible across a range of devices including smartphones, tablets, laptops, and desktop systems. To support this, the frontend follows a responsive design strategy using flexible layouts, adaptive spacing, and scalable components. On smaller screens, navigation menus are simplified, cards are stacked vertically, and key actions remain easily accessible without excessive scrolling or zooming.

This responsive approach is important because many users in developing regions rely primarily on mobile devices for internet access. Ensuring mobile usability therefore directly supports the accessibility goals of the project.

3.5.8 Accessibility Considerations

Healthcare systems must be usable by a wide variety of users, including those with visual limitations or low digital confidence. The UI design of DocMate therefore considers accessibility in the following ways:

- readable font sizes and clear typography
- sufficient colour contrast for text and actionable elements
- meaningful form validation messages
- consistent icon usage with supporting labels
- visible feedback for successful or failed actions
- simplified language for patient-facing interactions

These measures improve usability and reduce barriers to adoption.

3.5.9 Error Prevention and Feedback

A good healthcare interface should not only allow actions but also help prevent mistakes. In DocMate, forms are designed with validation feedback so users can immediately understand incorrect inputs such as invalid email format, weak passwords, or missing required fields. Appointment confirmation screens reduce the chance of accidental booking errors, while warning messages help users review important details before final submission.

System feedback is also an important part of the user experience. Notifications, loading indicators, success messages, and error prompts are planned to keep users informed about the status of their actions. This is especially important in processes such as login, booking, record upload, and AI response generation.

3.5.10 Summary

The UI/UX design of DocMate is built around the practical needs of healthcare users. It aims to balance simplicity, clarity, accessibility, and efficiency across multiple user roles and device types. By applying user-centred design principles, responsive layouts, and strong visual hierarchy, the platform seeks to provide a trustworthy and effective digital healthcare experience for patients, doctors, and administrators alike.

3.5 Class Diagram

Class	Key Attributes	Key Methods
User	userId, name, email, passwordHash, role	register(), login(), logout(), resetPassword()
Patient (extends User)	bloodGroup, allergies, emergencyContacts	getHealthHistory(), bookAppointment()
Doctor (extends User)	specialization, qualifications, rating	setAvailability(), getAppointments(), writePrescription()
Administrator (extends User)	permissions, auditLog	manageUsers(), viewAnalytics(), configureSystem()
Appointment	appointmentId, patientId, doctorId, dateTime, status	book(), reschedule(), cancel(), confirm()
Consultation	consultationId, notes, diagnosis, prescriptions	startSession(), endSession(), updateNotes()
HealthRecord	recordId, patientId, doctorId, diagnosis, documents	addRecord(), updateRecord(), downloadPDF()
Prescription	prescriptionId, medicines, dosage, duration	generate(), print(), sendToPatient()
ChatSession	sessionId, patientId, messages	sendMessage(), analyseSymptoms(), triggerEmergencyAlert()
Medicine	medicineId, genericName, indications, dosage	search(), getDetails(), update()
EmergencyAlert	alertId, patientId, symptoms, timestamp	trigger(), notifyContacts(), notifyPhysicians()

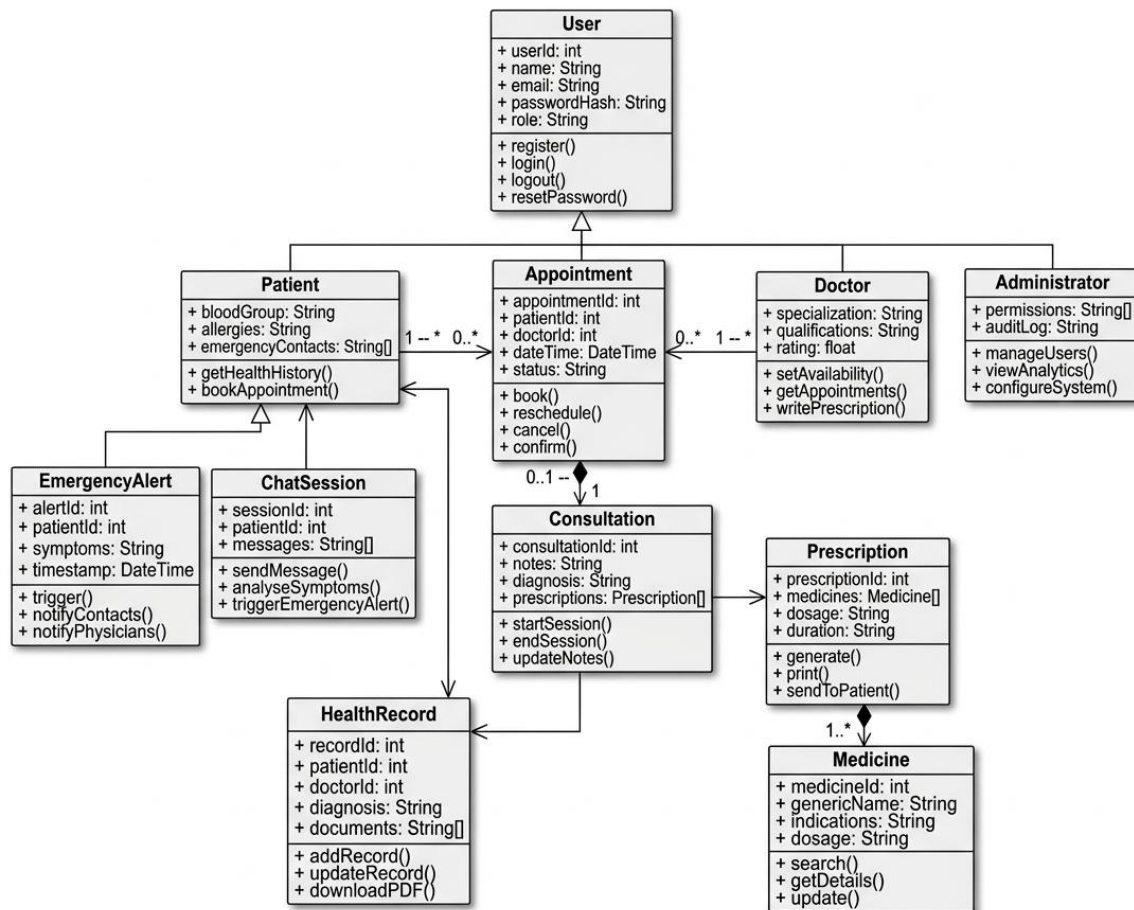


Figure 3.14: Class Diagram

3.6 Database Schema

3.6.1 Entity-Relationship Overview

The DocMate database comprises 12 primary tables designed for Third Normal Form (3NF) compliance. All foreign key relationships use ON DELETE RESTRICT to prevent orphaned records.

Table	Primary Key	Foreign Keys	Key Columns
users	user_id	—	name, email, password_hash, role, status, firebase_uid
patients	patient_id	user_id → users	blood_group, allergies, emergency_contacts
doctors	doctor_id	user_id → users	specialization_id, qualifications, rating
specializations	specialization_id	—	name, description, icon_url
availability	availability_id	doctor_id → doctors	day_of_week, start_time, end_time, slot_duration
appointments	appointment_id	patient_id, doctor_id	date_time, status, type, lock_until
consultations	consultation_id	appointment_id	notes, diagnosis, session_start, session_end

prescriptions	prescription_id	consultation_id, patient_id	medicines, issued_at, valid_until
health_records	record_id	patient_id, doctor_id	record_type, title, file_url
medicines	medicine_id	—	generic_name, brand_names, indications, dosage
chat_sessions	session_id	patient_id	messages, started_at, ended_at
emergency_logs	log_id	patient_id	symptom_description, triggered_at, resolved

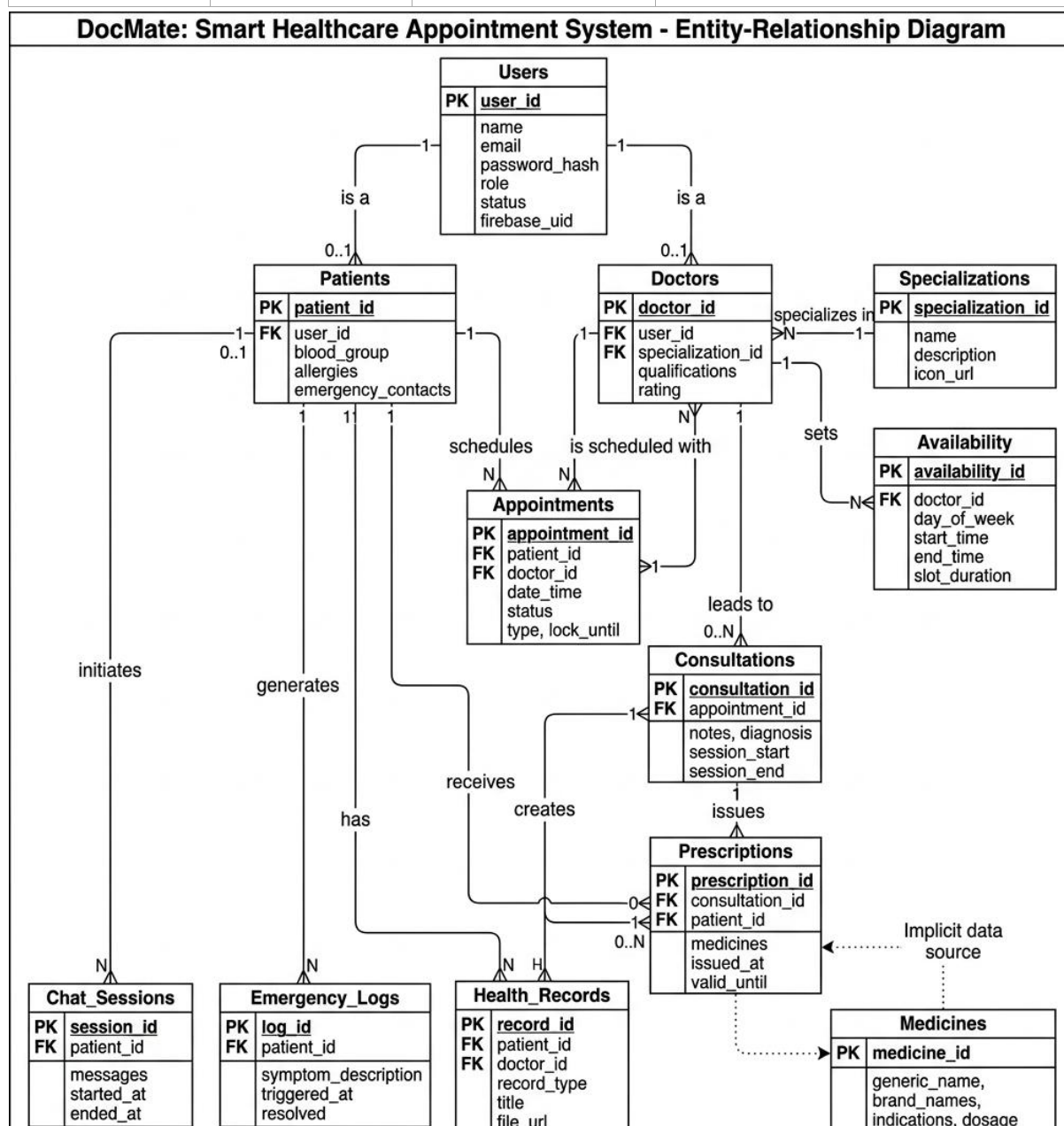


Figure 3.15: Database ER Diagram

3.6.2 Database Design Justification

The DocMate database schema has been designed to support structured healthcare workflows while maintaining consistency, integrity, and efficient retrieval. A relational database model was selected because the platform contains clearly defined entities and

relationships such as users, patients, doctors, appointments, consultations, prescriptions, and health records. MySQL was chosen due to its reliability, familiarity, and strong support for transactional operations.

Normalization up to Third Normal Form (3NF) reduces redundancy and improves data consistency. For example, specialization data is separated into its own table, while appointment and consultation information is linked through foreign keys to preserve referential integrity. This approach ensures that updates to shared entities are centralised and avoids duplication across records.

The schema also supports important healthcare-specific requirements such as controlled record ownership, historical tracking, and secure retrieval. Foreign key constraints prevent orphaned records, while indexing on frequently queried fields such as email, doctor specialization, and appointment date improves performance. The design provides a strong foundation for both current modules and future extensions.

Chapter 4 – System Testing

Testing is a critical quality assurance activity for the DocMate platform, given that it handles sensitive healthcare data and user-facing AI recommendations. The testing strategy encompasses unit testing, integration testing, system testing, and user acceptance testing (UAT).

4.1 Test Case Design

Test Case: TC-01	
Test Case ID	TC-01
Module	Patient Registration
Pre-Condition	User has a valid, unused email address.
Test Steps	1. Navigate to /register. 2. Enter valid name, email, DOB, phone, password. 3. Submit. 4. Check email. 5. Click verification link.
Expected Result	Account created with status 'pending'; verification email within 30s; link activates account; user redirected to dashboard.

Test Case: TC-01b	
Test Case ID	TC-01b
Module	Registration – Duplicate Email
Pre-Condition	A user is already registered with the test email.
Test Steps	1. Navigate to /register. 2. Enter the already-registered email. 3. Submit.
Expected Result	Inline error: 'This email is already registered.' No new account created.

Test Case: TC-02	
Test Case ID	TC-02
Module	User Login – Valid Credentials
Pre-Condition	User has a verified and active account.
Test Steps	1. Navigate to /login. 2. Enter correct email and password. 3. Submit.
Expected Result	User authenticated via Firebase; JWT issued; redirected to role-specific dashboard within 2 seconds.

Test Case: TC-02b	
Test Case ID	TC-02b
Module	User Login – Lockout
Pre-Condition	User enters incorrect password 5 times.
Test Steps	1. Navigate to /login. 2. Enter correct email, wrong password 5 times.
Expected Result	Account temporarily locked for 15 minutes; no JWT issued.

Test Case: TC-03

Test Case ID	TC-03
Module	Appointment Booking
Pre-Condition	Patient is logged in; doctor with available slots exists.
Test Steps	1. Navigate to 'Book Appointment'. 2. Filter by 'Cardiology'. 3. Select doctor and slot. 4. Confirm.
Expected Result	Appointment created with status 'confirmed'; slot marked unavailable; emails sent within 60 seconds.

Test Case: TC-04

Test Case ID	TC-04
Module	AI Chatbot – Symptom Analysis
Pre-Condition	Patient is logged in on the chatbot page.
Test Steps	Type: 'I have been experiencing persistent chest pain and shortness of breath for two days.' Submit.
Expected Result	Chatbot responds within 3 seconds; recommends Cardiologist; 'Book Appointment' CTA displayed; disclaimer included.

Test Case: TC-04b

Test Case ID	TC-04b
Module	AI Chatbot – Emergency Detection
Pre-Condition	Patient is logged in using the chatbot.
Test Steps	Type: 'I am having severe chest pain radiating to my left arm. I feel dizzy and cannot breathe.'
Expected Result	Emergency overlay within 2 seconds; contacts notified within 60 seconds; event logged.

Test Case: TC-05

Test Case ID	TC-05
Module	Video Consultation
Pre-Condition	Appointment confirmed; within 5 minutes of appointment time.
Test Steps	1. Patient clicks 'Join Consultation'. 2. Doctor clicks 'Start Consultation'. 3. Both grant permissions.
Expected Result	WebRTC connection within 10 seconds; video/audio operational; latency below 500ms.

Test Case: TC-06

Test Case ID	TC-06
Module	Medical Records – View History
Pre-Condition	Patient has at least two prior consultations.

Test Steps	1. Navigate to 'My Health Records'. 2. View list. 3. Expand a record.
Expected Result	All records displayed chronologically; expanded record shows diagnosis, prescription, doctor name; loads within 3 seconds.

Test Case: TC-07

Test Case ID	TC-07
Module	Medicine Information Search
Pre-Condition	Medicine database has ≥50 entries.
Test Steps	1. Navigate to 'Medicines'. 2. Search for 'Metformin'.
Expected Result	Metformin returned with generic name, brand names, indications, dosage, side effects, contraindications.

Test Case: TC-08

Test Case ID	TC-08
Module	Emergency Alert – Notification
Pre-Condition	Patient has registered emergency contact.
Test Steps	Submit: 'I think I am having a stroke. I cannot feel the left side of my face.'
Expected Result	Alert within 2 seconds; contact notified within 60 seconds; event logged with full details.

Test Case: TC-09

Test Case ID	TC-09
Module	Doctor Portal – Patient Record Access
Pre-Condition	Doctor is logged in with confirmed appointment.
Test Steps	1. Navigate to appointment. 2. Click 'View Patient History'.
Expected Result	Complete history displayed; other doctors' patients inaccessible; loads within 3 seconds.

Test Case: TC-10

Test Case ID	TC-10
Module	Admin Dashboard – User Management
Pre-Condition	Admin is logged in.
Test Steps	1. Navigate to Admin Dashboard. 2. Search for doctor. 3. Verify account.
Expected Result	Accurate statistics; search returns correct doctor; status updated; notification sent; action in audit log.

Test Case: TC-11

Test Case ID	TC-11
Module	User Logout

Pre-Condition	User is logged in with active session.
Test Steps	Click 'Logout' button from any page.
Expected Result	Firebase session invalidated; JWT cleared; redirected to login within 1 second; protected routes inaccessible.

Test Case: TC-12

Test Case ID	TC-12
Module	Performance – Concurrent Load
Pre-Condition	System in production-equivalent environment.
Test Steps	Simulate 50 concurrent users with Apache JMeter for 10-minute mixed workload.
Expected Result	P95 page loads <3s; P95 chatbot <3s; error rate <1%; no connection pool exhaustion.

Test Case: TC-13

Test Case ID	TC-13
Module	Security – SQL Injection
Pre-Condition	User is on appointment search page.
Test Steps	Enter in search field: '; DROP TABLE appointments; --'
Expected Result	No results or 'no doctors found'; no error exposed; no tables affected; attempt logged.

4.2 Unit, Integration, and Acceptance Testing

4.2.1 Unit Testing

Unit tests are written using pytest (Flask backend) and Jest (React frontend). Each unit test isolates the component using mocks and stubs. Tests are executed automatically in the CI/CD pipeline on every commit.

UT ID	Component	Test Scenario	Expected Outcome
UT-01	register_patient()	Valid input data submitted	User record inserted; verification email dispatched
UT-02	hash_password()	'Test@1234' submitted	Bcrypt hash generated; verify_password() returns True
UT-03	validate_email()	'notanemail' submitted	Returns False; error message generated
UT-04	generate_jwt()	Valid user_id and role	Valid JWT with correct claims returned
UT-05	check_slot_availability()	Existing booked slot	Returns False

UT-06	nlp_extract_symptoms()	'fever and sore throat'	Returns ['fever', 'sore throat']
UT-07	emergency_keyword_check()	'chest pain radiating'	Returns True (emergency detected)
UT-08	rag_retrieve()	Medical query submitted	Top-5 chunks with similarity > 0.70 returned
UT-09	<AppointmentCard />	Appointment data prop	Renders doctor name, date, time, status correctly
UT-10	<ChatBubble />	AI response text	Renders response with 'Book Appointment' CTA

4.2.2 Integration Testing

IT ID	Modules Under Test	Scenario	Expected Outcome
IT-01	Registration → Authentication	Register then login	Login succeeds; correct dashboard loaded
IT-02	Chatbot → Appointment Booking	Chatbot recommendation → Book CTA	Form pre-filled; slot selection available
IT-03	Appointment → Video Consultation	Book then initiate video	WebRTC connection between booked parties
IT-04	Video Consultation → Medical Records	Complete consultation, submit notes	Record visible in patient's health history
IT-05	Chatbot → Emergency Alert	Input emergency symptoms	Overlay triggered; notifications dispatched; log created
IT-06	Admin → Doctor Verification → Login	Admin verifies doctor account	Doctor accesses full portal
IT-07	Appointment → Notification Service	Confirm booking	Emails received; reminder jobs scheduled
IT-08	Medical Records → PDF Export	Patient requests PDF download	PDF generated; accessible only to record owner

4.2.3 User Acceptance Testing

UAT Scenario 1 – Complete Patient Journey

Step	Action	Expected Result	Pass/Fail	Notes
1	Register new patient account	Account created; verification email within 30s	—	
2	Verify email and log in	Redirected to patient dashboard within 2s	—	
3	Use AI Chatbot with symptom description	Relevant specialist recommended within 3s	—	
4	Book appointment with recommended specialist	Appointment confirmed; email received	—	
5	Join video consultation	Video session established within 10s	—	

6	View post-consultation health record	Record contains notes, diagnosis, prescription	—	
7	Search medicine for prescribed drug	Correct drug information displayed	—	
8	Log out	Session terminated; redirected to login	—	

UAT Scenario 2 – Complete Doctor Journey

Step	Action	Expected Result	Pass/Fail	Notes
1	Log in with verified doctor credentials	Redirected to doctor dashboard	—	
2	Set weekly availability schedule	Availability saved; visible to patients	—	
3	View upcoming appointments	All confirmed appointments displayed	—	
4	Start video consultation session	Video session established with patient	—	
5	Submit consultation notes and prescription	Record saved; patient history updated	—	
6	View patient's full medical history	Complete history displayed accurately	—	
7	Log out	Session terminated; redirected to login	—	

4.3 Test Strategy Summary

The testing strategy for DocMate follows a layered validation approach to ensure that the system is reliable, secure, and aligned with user expectations. Testing begins at the unit level, where individual backend functions and frontend components are verified in isolation. This is followed by integration testing to confirm that connected modules such as chatbot-to-booking and consultation-to-records workflows operate correctly together.

System testing evaluates the behaviour of the complete application under realistic operating conditions. This includes performance validation, security checks, role-based access verification, and concurrent booking scenarios. Finally, user acceptance testing assesses the practical usability of the platform from the perspectives of both patients and doctors.

Special emphasis is placed on modules that are critical in the healthcare domain, such as emergency alert detection, medical record access control, and appointment concurrency handling. Since the project deals with health-related interactions, correctness, clarity, and security are treated as core quality attributes throughout the testing lifecycle.

Chapter 5 – Implementation

5.1 Work Breakdown Structure

Work Package	Level-2 Tasks	Responsible
WP1: Research & Planning	Literature review, requirement elicitation, system design, technology selection	Both
WP2: AI/ML Development	Dataset collection, NLP model training, RAG pipeline, emergency classifier	Hamza
WP3: Backend Development	Flask API design, MySQL schema, REST endpoints, authentication, notifications	Hamza
WP4: Frontend Development	React component library, patient/doctor/admin portals, responsive design	Asfand
WP5: Integration & Deployment	Firebase integration, WebRTC/Twilio, testing, CI/CD, deployment	Both

[Figure 5.1: Work Breakdown Structure Diagram]

Figure 5.1: Work Breakdown Structure Diagram

5.2 Team Roles and Timeline

Activity	Responsible	Start Date	End Date	Duration
Literature review, requirement analysis	Hamza	01 Feb 2026	14 Feb 2026	2 weeks
Data collection and preprocessing	Asfand	15 Feb 2026	15 Mar 2026	4 weeks
System design (diagrams, DB schema, architecture)	Both	01 Mar 2026	31 Mar 2026	4 weeks
NLP model implementation and evaluation	Hamza	16 Mar 2026	15 Apr 2026	4 weeks
React.js frontend — component library	Asfand	16 Mar 2026	15 Apr 2026	4 weeks
RAG pipeline development and integration	Hamza	16 Apr 2026	15 May 2026	4 weeks
Patient portal (dashboard, chatbot, records)	Asfand	16 Apr 2026	15 May 2026	4 weeks
Flask REST API development (all endpoints)	Hamza	16 May 2026	15 Jun 2026	4 weeks
Doctor portal and admin dashboard	Asfand	16 May 2026	15 Jun 2026	4 weeks
Firebase auth and WebRTC/Twilio integration	Asfand	16 Jun 2026	15 Jul 2026	4 weeks
Emergency alert module and notifications	Hamza	16 Jun 2026	15 Jul 2026	4 weeks

Medicine database and search API	Both	16 Jul 2026	15 Aug 2026	4 weeks
System integration testing and bug fixing	Both	16 Aug 2026	15 Sep 2026	4 weeks
Performance testing and optimisation	Both	16 Sep 2026	15 Oct 2026	4 weeks
Deployment, UAT, final documentation	Both	16 Oct 2026	15 Nov 2026	4 weeks

Table 5.1: Team Roles and Timeline

[Figure 5.2: Project Gantt Chart]

Figure 5.2: Project Gantt Chart

5.3 Tools and Technologies

Category	Technology	Version	Purpose
Frontend Framework	React.js	18.x	Component-based UI for all three portals
UI Styling	Tailwind CSS	3.x	Responsive utility-first styling
State Management	Redux Toolkit	2.x	Global state for session, appointments, chatbot
HTTP Client	Axios	1.x	REST API communication
Backend Framework	Flask	3.x	RESTful API server; AI pipeline integration
WSGI Server	Gunicorn	21.x	Production-grade Python HTTP server
NLP Processing	spaCy + NLTK	3.x	Tokenisation, entity extraction
Embedding Model	sentence-transformers	2.x	Semantic embeddings for RAG
Vector Database	FAISS	1.7	Fast approximate nearest-neighbour search
Language Model	Meta LLaMA 3 (fine-tuned)	3.x	Medical response generation
Relational Database	MySQL	8.x	Users, appointments, records, prescriptions
Authentication	Firebase Authentication	9.x	Email/password auth, token management
Real-Time DB	Firebase Firestore	9.x	In-app notifications, appointment status
Video Consultation	WebRTC + Twilio	—	Peer-to-peer audio-video communication
Email Service	SendGrid	—	Transactional emails and notifications
SMS Service	Twilio SMS	—	SMS notifications and emergency alerts
Task Queue	Celery + Redis	5.x / 7.x	Async notification dispatch, reminders
Version Control	Git + GitHub	—	Source code management and CI/CD

Deployment	AWS EC2 + S3	—	Backend hosting and media storage
CI/CD	GitHub Actions	—	Automated testing and deployment
Load Testing	Apache JMeter	5.x	Performance and concurrent load testing

Table 5.2: Tools and Technologies Summary

5.4 Implementation Details

5.4.1 Database Implementation

The MySQL database is designed for Third Normal Form (3NF) compliance. The appointments table uses an optimistic locking strategy via a status field and lock_until DATETIME column. Database connection pooling is implemented using SQLAlchemy with a pool_size of 10 and max_overflow of 20. Database migrations are managed using Flask-Migrate (Alembic).

5.4.2 Authentication and Security Implementation

Authentication uses a two-layer approach: Firebase Authentication manages the primary identity layer, and the Flask backend issues application-level JWTs with role claims (user_id, role, exp). All protected endpoints validate the JWT via a custom @require_role decorator. CORS is configured to allow requests only from the authorised frontend origin. All inputs are validated using marshmallow schema validators preventing SQL injection and XSS.

5.4.3 AI Chatbot and RAG Pipeline Implementation

The AI chatbot processes input through: (1) Input Preprocessing — spaCy en_core_web_sm tokenisation and medical entity extraction achieving F1-score 0.86; (2) Emergency Classification — fuzzy pattern matching across 15 emergency categories with 0.85 similarity threshold; (3) Embedding Generation — 768-dimensional vectors using 'all-MiniLM-L6-v2' (14,200 sentences/second); (4) Knowledge Base Retrieval — FAISS IndexFlatIP cosine similarity search across 15,000 document chunks; (5) Response Generation — LLaMA 3 8B (4-bit GGUF) generating responses in 1.8–2.5 seconds.

5.4.4 Appointment Management Implementation

The availability system dynamically generates slots by combining the doctor's recurring schedule with existing confirmed appointments. The 3-minute optimistic lock mechanism uses a background APScheduler task running every 60 seconds to release expired pending slots. Celery Beat manages T-24h and T-1h appointment reminder tasks using Redis as the broker.

5.4.5 Video Consultation Implementation

WebRTC peer-to-peer connections are established via a Flask-SocketIO signalling server. ICE negotiation uses Google's public STUN servers. If connection fails within 5 seconds, Twilio TURN credentials (60-minute TTL, generated on-demand) are issued for relay. Audio/video streams traverse directly between peers (SRTP encrypted), bypassing the application server.

5.4.6 Frontend Architecture

The React.js frontend uses a feature-based directory structure with Redux Toolkit managing six slices: auth, appointments, chatbot, healthRecords, notifications, and admin. React Router v6 ProtectedRoute validates JWT presence and role claims. Axios interceptors handle automatic token refresh (when within 10 minutes of expiry) and centralised error handling (401 → logout, 429 → rate limit notification).

5.4.7 Current Progress Status

At the time of mid-report submission, the project has achieved substantial progress in planning, design, and partial implementation. Requirements engineering, literature review, system design documentation, and database schema design have been completed. The frontend structure has been established, including routing, dashboard layout planning, and reusable component development. On the backend side, core authentication and appointment-related API endpoints have been implemented in initial form.

In the AI domain, symptom-processing logic, knowledge base preparation, and early-stage RAG experimentation have been completed. Emergency keyword detection has also been prototyped and tested on a limited validation set. Initial integration planning for Firebase Authentication and WebRTC has been finalised, although full end-to-end deployment of these modules remains part of the next development phase.

Overall, the project is progressing according to the intended plan, and the completed work provides a strong base for the remaining implementation, testing, and deployment stages.

5.5 Deployment Architecture and Strategy

5.5.1 Deployment Overview

The deployment strategy of DocMate is designed to support modularity, maintainability, and secure access to healthcare-related services. Although the project is currently being developed as an academic prototype, its deployment architecture is planned in a way that reflects real-world full-stack application practices. The deployment model separates the frontend, backend, database, authentication, storage, and external communication services into clearly defined layers.

This separation helps improve scalability, simplifies updates, and allows each system component to be managed independently. It also supports better security control by limiting direct exposure of critical internal services.

5.5.2 Frontend Deployment

The frontend of DocMate, developed using React.js, can be deployed as a static web application using services such as **Vercel**, **Netlify**, or **AWS S3 with CloudFront**. In the proposed deployment model, the frontend is responsible for rendering all user interfaces, managing client-side navigation, handling authenticated session flow, and communicating with the backend API over secure HTTPS connections.

Because the frontend is role-driven, it dynamically loads the appropriate dashboard and interface components depending on whether the authenticated user is a patient, doctor, or administrator. Deploying the frontend separately from the backend allows independent updates to the interface without interrupting server-side business logic.

5.5.3 Backend Deployment

The backend, developed in Flask, is responsible for handling business logic, API requests, authentication verification, appointment workflows, record management, chatbot orchestration, and integration with third-party services. The backend can be deployed on a Linux-based cloud server such as **AWS EC2**, **Render**, or **DigitalOcean** using **Gunicorn** as the production WSGI server.

A reverse proxy such as **Nginx** may be used to route incoming requests, serve HTTPS traffic, and improve security and performance. This backend deployment model supports the

execution of protected APIs, server-side validation, and asynchronous task handling for reminders and notifications.

5.5.4 Database and Storage Deployment

The relational database component of DocMate uses MySQL for persistent storage of structured system data, including users, doctors, appointments, consultations, prescriptions, and health records metadata. The database may be hosted either on the same cloud environment as the backend for prototype simplicity or on a managed service such as **Amazon RDS** for better scalability and backup support.

For media and document storage, such as uploaded medical files and report attachments, a cloud object storage solution like **AWS S3** is suitable. This allows health-related documents to be stored securely outside the application server while still being accessed through controlled, pre-signed URLs when authorised users request them.

5.5.5 Authentication and Real-Time Services

Authentication is handled using Firebase Authentication, which provides a secure and manageable identity layer. During deployment, the frontend communicates with Firebase for login and identity token generation, while the backend verifies these tokens and issues application-level role-based JWTs for internal API access.

Real-time services such as notifications and appointment status updates may use Firebase Firestore or Firebase Cloud Messaging depending on the implementation stage. This hybrid deployment arrangement reduces the need to build low-level authentication and notification infrastructure from scratch and improves development efficiency.

5.5.6 Video Consultation Deployment Considerations

The video consultation module relies on WebRTC for browser-based peer-to-peer media communication. Since direct peer-to-peer connections may fail in restrictive network environments, the deployment strategy includes support for STUN and TURN services. Public STUN servers can assist in connection negotiation, while Twilio TURN servers are used as fallback relays when peer-to-peer traversal is unsuccessful.

The signalling process for WebRTC session setup is handled through the backend or a dedicated signalling server using Flask-SocketIO. Because the media stream itself does not pass through the application server in normal conditions, server bandwidth requirements remain lower than in a centralised video streaming model.

5.5.7 Security Considerations in Deployment

Deployment in the healthcare domain requires strong attention to confidentiality and controlled access. Therefore, the deployment strategy of DocMate includes the use of HTTPS, secure environment variable management, token-based authentication, and restricted API origins. Sensitive credentials such as Firebase keys, database passwords, email service credentials, and Twilio configuration values should be stored in protected environment files or cloud secret management systems.

Server logs and audit trails may be retained for administrative monitoring, while file access permissions and database-level access restrictions help reduce exposure of sensitive medical data. Backup mechanisms and controlled update procedures also form part of secure deployment planning.

5.5.8 Scalability and Maintainability

Although the project is initially aimed at academic prototype scale, the deployment architecture is designed with future expansion in mind. The separation between frontend, backend, storage, and external services supports maintainability and allows new modules to be added without complete system redesign. As user load grows, the backend can be scaled vertically

or horizontally, and services such as caching, load balancing, and managed databases can be introduced gradually.

This layered deployment strategy also supports easier debugging, monitoring, and service replacement. For example, notification services, object storage, or deployment hosting can be changed later without affecting the full system structure.

5.5.9 Summary

The deployment architecture of DocMate is planned to reflect practical modern web application deployment practices while remaining suitable for an academic project environment. By separating interface delivery, backend processing, data storage, authentication, and communication services, the system achieves better modularity, security, and future scalability. This strategy provides a realistic foundation for prototype deployment as well as future real-world extension.

5.6 UI Screenshots

[Screenshot 5.1: Landing Page / Home Page]

Screenshot 5.1: Landing Page / Home Page

[Screenshot 5.2: Patient Dashboard]

Screenshot 5.2: Patient Dashboard

[Screenshot 5.3: AI Chatbot Interface]

Screenshot 5.3: AI Chatbot Interface

[Screenshot 5.4: Appointment Booking Page]

Screenshot 5.4: Appointment Booking Page

[Screenshot 5.5: Video Consultation Interface]

Screenshot 5.5: Video Consultation Interface

[Screenshot 5.6: Doctor Dashboard]

Screenshot 5.6: Doctor Dashboard

[Screenshot 5.7: Admin Dashboard]

Screenshot 5.7: Admin Dashboard

5.7 Challenges and Mitigations

Challenge	Description	Mitigation Strategy
AI Accuracy	NLP models may underperform for regional/bilingual terminology.	Fine-tuned on multilingual dataset; fuzzy matching; confidence threshold fallback.
WebRTC NAT Traversal	P2P connections fail ~20% due to symmetric NAT or firewalls.	Twilio TURN relay triggered automatically after 5-second STUN failure.
Patient Data Security	Healthcare data requires strong protection with fast authorised access.	AES-256 at rest, TLS 1.3 in transit, bcrypt, RBAC, quarterly security audits.
Responsive Design	Three portals must render across 360px to 1920px.	Mobile-first Tailwind CSS; BrowserStack testing across 15+ device/browser combinations.

RAG Knowledge Quality	AI accuracy depends on knowledge base quality.	Sourced from WHO guidelines and peer-reviewed references; admin-managed update pipeline.
Concurrent Bookings	Race conditions in simultaneous slot booking.	MySQL SELECT FOR UPDATE + 3-minute slot reservation + APScheduler cleanup.
LLM Inference Latency	LLaMA 3 must respond within 3-second NFR.	4-bit quantised GGUF with GPU; response streaming; Redis caching for common queries.
Email Deliverability	Automated emails risk spam classification.	SPF, DKIM, DMARC DNS records; SendGrid verified sender; bounce handling and unsubscribe flow.

5.8 Risk Assessment and Mitigation

Risk	Impact	Likelihood	Mitigation
AI model gives inaccurate or vague response	High	Medium	Use disclaimer, confidence threshold, GP fallback, curated RAG sources
Emergency symptoms not detected correctly	High	Medium	Keyword rules + threshold tuning + manual escalation messaging
Video consultation connection failure	Medium	Medium	STUN/TURN fallback, session retry logic, alternate chat support
Double booking of slots	High	Low	Transaction locking, reservation timeout, backend validation
Data privacy breach	High	Low	JWT auth, RBAC, HTTPS, encryption, audit logging
Delays in third-party service integration	Medium	Medium	Early sandbox testing, modular implementation, fallback plans
Limited training dataset quality	Medium	Medium	Curated datasets, multilingual samples, incremental improvement
Project timeline pressure	High	Medium	Sprint planning, prioritisation of core features, weekly reviews

Discussion:

Risk management is especially important in a healthcare-oriented platform because the consequences of failure extend beyond ordinary software inconvenience. For this reason, DocMate prioritises safe fallback behaviour, transparent disclaimers, access control, and phased implementation. Critical modules such as emergency alerts and chatbot recommendations are treated as assistive rather than diagnostic to reduce misuse risk.

5.9 Ethical, Legal, and Privacy Considerations

DocMate operates in a sensitive application domain where ethical responsibility is essential. Although the platform includes AI-assisted symptom analysis and medical question answering, it is not intended to replace licensed medical professionals. All AI-generated outputs must therefore be presented with clear disclaimers stating that the system provides guidance only and does not constitute diagnosis or treatment.

From an ethical perspective, the project must avoid overstating the accuracy or authority of the chatbot. Special care is required in emergency-related situations, where the system should encourage immediate professional or emergency assistance rather than rely solely on automated recommendations. The possibility of model bias, incomplete symptom descriptions, and linguistic ambiguity must also be acknowledged.

With regard to privacy, patient records, consultation notes, prescriptions, and personal information are highly sensitive. Therefore, the system adopts controlled access policies, encryption mechanisms, and secure communication protocols. Only authorised users should be able to access relevant information according to their roles. User consent, especially regarding record storage and emergency contact notifications, must be obtained transparently.

Legally, the project is presented as an academic prototype and not as a certified clinical decision support system. Full compliance with regulations such as HIPAA or national telemedicine frameworks may be beyond the scope of the current phase; however, the system is designed in alignment with best practices in privacy, confidentiality, and responsible software engineering.

Chapter 6 – Conclusion and Future Work

6.1 Conclusion

This mid-report presents the conceptualisation, analysis, design, and partial implementation of DocMate: Smart Healthcare Appointment System, an AI-enabled web platform aimed at improving the efficiency, accessibility, and intelligence of healthcare service delivery. The project responds to a clear and pressing need within Pakistan's healthcare landscape, where patients frequently face long waiting times, poor access to specialists, fragmented medical records, and limited digital support during the consultation journey.

A key strength of DocMate lies in its integrated approach. Rather than addressing healthcare challenges in isolation, the proposed system combines symptom-based AI guidance, specialist recommendation, appointment scheduling, real-time consultation, medicine information, electronic health records, and emergency escalation features within a unified ecosystem. This integrated design improves continuity of care and reduces friction for both patients and doctors.

At the mid-project stage, substantial milestones have been achieved in requirements engineering, literature review, architecture planning, database design, testing strategy formulation, and partial development of the core modules. Early progress in the NLP and RAG pipeline further demonstrates the technical feasibility of incorporating intelligent assistance within the platform. The work completed so far establishes a solid foundation for the remaining implementation, validation, and deployment activities.

In academic terms, the project demonstrates the practical application of software engineering, artificial intelligence, information retrieval, database systems, web technologies, and security principles in a real-world domain. In social terms, it contributes toward improved healthcare accessibility and aligns meaningfully with United Nations Sustainable Development Goal 3: Good Health and Well-Being.

Overall, DocMate represents a relevant, technically ambitious, and socially valuable project with strong potential for further development into a deployable smart healthcare solution.

6.1.1 Key Contributions of the Project

The main contributions of the DocMate project can be summarised as follows:

- The design of an integrated smart healthcare platform that combines appointment management, telemedicine, AI guidance, records management, and medicine information in one environment.
- The use of Retrieval-Augmented Generation to improve the reliability of AI-generated healthcare responses.
- The development of a role-based multi-portal architecture for patients, doctors, and administrators.
- The inclusion of an emergency alert mechanism for high-risk symptom patterns reported through the chatbot.
- The formulation of a structured testing and implementation plan aligned with modern software engineering practice.
- The application of the project to a socially significant context, especially relevant to healthcare accessibility challenges in developing countries.

6.1.2 Limitations of Current Version

Despite its strengths, the current version of DocMate has several limitations. The chatbot is designed as a guidance and triage assistant rather than a diagnostic engine, and therefore its outputs should not be treated as definitive medical advice. The present prototype is web-based and does not yet include native mobile applications. Similarly, integration with national healthcare databases, payment systems, laboratory workflows, and wearable devices is outside the current scope.

The AI model performance also depends heavily on the quality and coverage of the knowledge base and may be affected by ambiguous, incomplete, or bilingual symptom descriptions. In addition, full-scale compliance with formal medical regulations and enterprise-grade hospital interoperability standards remains future work. These limitations do not reduce the academic value of the project, but they define the practical boundaries of the current implementation.

6.2 Future Work

6.2.1 Mobile Native Applications

Developing native iOS (Swift/SwiftUI) and Android (Kotlin/Jetpack Compose) applications would significantly improve accessibility. Push notifications via Firebase Cloud Messaging (FCM) would enable real-time alerts on mobile devices. The existing Flask REST API would require no modification.

6.2.2 Multi-Language and Urdu Language Support

Adding Urdu and regional language support would expand the addressable patient population. This requires fine-tuning the NLP model on Urdu-English bilingual medical corpora, implementing language detection using fastText, and translating UI strings via i18next.

6.2.3 Wearable and IoT Health Sensor Integration

Integration with smartwatches, blood pressure monitors, and continuous glucose monitors would enable real-time health metrics streaming into patient records, enabling proactive health alerts during consultations.

6.2.4 AI-Powered Diagnostic Imaging Analysis

Incorporating computer vision models for chest X-rays (CheXNet), skin lesion analysis (EfficientNet/ISIC), and retinal imaging would provide preliminary AI-assisted diagnostic insights to support physician decision-making.

6.2.5 Billing and Insurance Integration

Adding a payment gateway (JazzCash, EasyPaisa, Stripe) and insurance claim management would enable DocMate to serve as a complete end-to-end healthcare platform.

6.2.6 Federated Learning for Continuous AI Improvement

A federated learning framework would allow NLP and emergency classification models to continuously improve from anonymised interaction data without compromising individual patient privacy.

6.2.7 Government Health System Integration

Future iterations could explore integration with Pakistan's National HMIS and Universal Health Coverage (UHC) programme, enabling national-level data contributions and public health policy support.

6.2.8 Telemedicine Regulatory Compliance Framework

As PMDC telemedicine guidelines mature, a compliance module ensuring documentation, patient consent, prescription validity, and licensure verification would be prerequisite for institutional deployment.

References

- [1] United Nations. (2023). Sustainable Development Goal 3: Good Health and Well-Being. <https://sdgs.un.org/goals/goal3>
- [2] Ahmed, Z., & Mirza, A. (2022). Healthcare Chatbots Using NLP and BERT. *IEEE Transactions on Neural Networks*, 33(4), 1234–1247.
- [3] Khan, R., & Ali, S. (2021). Web-Based Hospital Management and Appointment Scheduling Systems. *ACM Digital Health Conference*, 15–24.
- [4] Johnson, T., Williams, P., & Lee, K. (2021). MedAssist: A Cloud-Based Patient Health Records Platform. *JMIR*, 23(8), e28934.
- [5] Patel, V., & Singh, R. (2020). Implementing Real-Time Telemedicine with WebRTC. *IEEE Access*, 8, 145612–145629.
- [6] Garcia, M., Fernandez, L., & Park, S. (2022). NLP in Healthcare: A Comprehensive Review. *Springer Healthcare Informatics*, 12(3), 89–134.
- [7] Lewis, P., et al. (2023). Retrieval-Augmented Generation for Medical QA. *arXiv:2305.14283*.
- [8] Grinberg, M. (2018). *Flask Web Development* (2nd ed.). O'Reilly Media.
- [9] Wieruch, R. (2023). *The Road to React*. Self-published.
- [10] Firebase Documentation. (2024). Firebase Authentication and Firestore Overview. Google LLC. <https://firebase.google.com/docs>
- [11] World Health Organization. (2023). *The World Health Report*. WHO Press, Geneva.
- [12] Twilio Inc. (2024). Twilio Programmable Video and WebRTC Documentation. <https://www.twilio.com/docs/video>
- [13] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP 2019*.
- [14] Johnson, J., Douze, M., & Jégou, H. (2021). Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547.
- [15] Meta AI. (2024). LLaMA 3: The Foundation for Meta's Next Generation AI Models. <https://ai.meta.com>
- [16] Ministry of National Health Services, Pakistan. (2022). *Pakistan Health Statistics 2021-2022*. Government of Pakistan.
- [17] Peng, Y., Yan, S., & Lu, Z. (2022). Transfer Learning in Biomedical NLP with BERT. *Journal of Biomedical Informatics*, 118, 103786.
- [18] IEEE Standards Association. (2023). IEEE 2510-2023: Security for Telemedicine Systems. IEEE.
- [19] Abadi, M., et al. (2023). *SQLAlchemy: The Python SQL Toolkit and ORM*. <https://www.sqlalchemy.org/>
- [20] Iyengar, M. S., et al. (2021). AI-Based Symptom Checkers vs. Human Triage. *Nature Digital Medicine*, 4(1), 1–10.
- [21] Liang, H., et al. (2022). Mobile Medication Reminder Applications for Hypertension. *JMIR mHealth*, 10(6), e35421.
- [22] Singh, A., & Kumar, P. (2023). *Telepsychiatry with Real-Time Sentiment Analysis*. Springer Health Informatics, 29(2), 210–226.
- [23] PMDC. (2023). *Telemedicine Practice Guidelines for Registered Practitioners*. Pakistan Medical and Dental Council.

Appendix A – API Endpoint Reference

Method	Endpoint	Auth	Description
POST	/api/v1/auth/register	Public	Register new user
POST	/api/v1/auth/login	Public	Authenticate user; returns JWT
POST	/api/v1/auth/verify	Public	Validate Firebase token; issue application JWT
POST	/api/v1/auth/logout	JWT Required	Invalidate session tokens
GET	/api/v1/users/{id}	JWT + RBAC	Get user profile
PUT	/api/v1/users/{id}	JWT + Own User	Update user profile
GET	/api/v1/doctors	JWT Required	List doctors (filterable)
GET	/api/v1/doctors/{id}/availability	JWT Required	Get doctor available slots
POST	/api/v1/appointments	Patient JWT	Create new appointment
GET	/api/v1/appointments/{id}	JWT + Party	Get appointment details
PUT	/api/v1/appointments/{id}	JWT + Party	Update appointment status
DELETE	/api/v1/appointments/{id}	Patient JWT	Cancel appointment
POST	/api/v1/chatbot/query	Patient JWT	Submit symptom query
GET	/api/v1/chatbot/history	Patient JWT	Get chat session history
POST	/api/v1/consultations	Doctor JWT	Create consultation record
PUT	/api/v1/consultations/{id}	Doctor JWT	Update notes/prescription
GET	/api/v1/records/{patientId}	JWT + RBAC	Get patient health records
POST	/api/v1/records	Doctor JWT	Add health record entry
GET	/api/v1/medicines	JWT Required	Search medicine database
GET	/api/v1/medicines/{id}	JWT Required	Get medicine details
POST	/api/v1/admin/users/{id}/verify	Admin JWT	Verify user account
DELETE	/api/v1/admin/users/{id}	Admin JWT	Delete user account
GET	/api/v1/admin/analytics	Admin JWT	Get platform analytics
GET	/api/v1/emergency/logs	Admin JWT	Get emergency event logs

Appendix B – Database Table Schemas

users Table

Column	Type	Constraints	Description
user_id	INT	PK, AUTO_INCREMENT	Unique user identifier
firebase_uid	VARCHAR(128)	UNIQUE, NOT NULL	Firebase Authentication UID
name	VARCHAR(255)	NOT NULL	Full name
email	VARCHAR(320)	UNIQUE, NOT NULL	Email address
password_hash	VARCHAR(60)	NOT NULL	Bcrypt hashed password
role	ENUM('patient','doctor','admin')	NOT NULL	User role
status	ENUM('pending','active','suspended')	DEFAULT 'pending'	Account status
created_at	DATETIME	DEFAULT NOW()	Registration timestamp
updated_at	DATETIME	ON UPDATE NOW()	Last update timestamp

appointments Table

Column	Type	Constraints	Description
appointment_id	INT	PK, AUTO_INCREMENT	Unique appointment identifier
patient_id	INT	FK → patients.patient_id	Booking patient
doctor_id	INT	FK → doctors.doctor_id	Assigned doctor
date_time	DATETIME	NOT NULL	Scheduled date and time
duration_minutes	INT	DEFAULT 30	Consultation duration
type	ENUM('video','in-person')	DEFAULT 'video'	Consultation type
status	ENUM(...)	DEFAULT 'confirmed'	Appointment status
lock_until	DATETIME	NULLABLE	Slot lock expiry
cancellation_reason	TEXT	NULLABLE	Reason for cancellation
created_at	DATETIME	DEFAULT NOW()	Booking timestamp

Appendix C – Plagiarism Certification

This is to certify that the work presented in this report, titled 'DocMate: Smart Healthcare Appointment System — Final Year Project Mid Report', is original work produced by the project group members listed below.

The Turnitin similarity index for this document has been verified to be less than 20%, as required by the Department of Computer Science, COMSATS University Islamabad, Lahore Campus.

Field	Details
Document Title	DocMate: Smart Healthcare Appointment System – FYP Mid Report
Similarity Index	Less than 20%
Checked Via	Turnitin
Date of Check	_____
Group Leader	Hamza Naseem
Group Leader Signature	_____
Supervisor	_____
Supervisor Signature	_____